Final Project

This notebook is adapted from here: https://aiqm.github.io/torchani/examples/nnp_training.html

Data preparation

1. Create a working directory: `/global/scratch/users/[USER_NAME]/[DIR_NAME]`. Replace the [USER_NAME] with yours and specify a [DIR_NAME] you like.
2. Copy the Jupyter Notebook to the working directory
3. Download the ANI dataset `ani_dataset_gdb_s01_to_s04.h5` from bCourses and upload it to the working directory

In [1]:
```
import numpy as np
from tqdm import tqdm
import torch
import torch.nn as nn
import torchani
import matplotlib.pyplot as plt
```

Use GPU

In [2]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
```

```
cuda
```

Set up AEV computer

AEV: Atomic Environment Vector (atomic features)

Ref: Chem. Sci., 2017, 8, 3192

In [3]:
```
def init_aev_computer():
    Rcr = 5.2
```

```
    Rca = 3.5
    EtaR = torch.tensor([16], dtype=torch.float, device=device)
    ShfR = torch.tensor([
        0.900000, 1.168750, 1.437500, 1.706250,
        1.975000, 2.243750, 2.512500, 2.781250,
        3.050000, 3.318750, 3.587500, 3.856250,
        4.125000, 4.393750, 4.662500, 4.931250
    ], dtype=torch.float, device=device)


    EtaA = torch.tensor([8], dtype=torch.float, device=device)
    Zeta = torch.tensor([32], dtype=torch.float, device=device)
    ShfA = torch.tensor([0.90, 1.55, 2.20, 2.85], dtype=torch.float, device=device)
    ShfZ = torch.tensor([
        0.19634954, 0.58904862, 0.9817477, 1.37444680,
        1.76714590, 2.15984490, 2.5525440, 2.94524300
    ], dtype=torch.float, device=device)

    num_species = 4
    aev_computer = torchani.AEVComputer(
        Rcr, Rca, EtaR, ShfR, EtaA, Zeta, ShfA, ShfZ, num_species
    )
    return aev_computer

aev_computer = init_aev_computer()
aev_dim = aev_computer.aev_length
print(aev_dim)
```

```
384
```

Prepare dataset & split

In [4]:

```
def load_ani_dataset(dspath):
    self_energies = torch.tensor([
        0.500607632585, -37.8302333826,
        -54.5680045287, -75.0362229210
    ], dtype=torch.float, device=device)
    energy_shifter = torchani.utils.EnergyShifter(None)
    species_order = ['H', 'C', 'N', 'O']

    dataset = torchani.data.load(dspath)
```

```
    dataset = dataset.subtract_self_energies(energy_shifter, species_order)
    dataset = dataset.species_to_indices(species_order)
    dataset = dataset.shuffle()
    return dataset

dataset = load_ani_dataset("./ani_gdb_s01_to_s04.h5")
# Use dataset.split method to do split
train_data, val_data, test_data = dataset.split(0.8,0.1,0.1)
```

Batching

In [5]:
```
batch_size = 8192
# use dataset.collate(...).cache() method to do batching
train_data_loader = train_data.collate(batch_size).cache()
val_data_loader = val_data.collate(batch_size).cache()
test_data_loader = test_data.collate(batch_size).cache()
```

In [6]:
```
total_train = sum(len(batch['energies']) for batch in train_data_loader)
total_val = sum(len(batch['energies']) for batch in val_data_loader)
total_test = sum(len(batch['energies']) for batch in test_data_loader)

print(f"Train: {total_train}")
print(f"Val:   {total_val}")
print(f"Test:  {total_test}")
print(f"Total: {total_train + total_val + total_test}")
```

```
Train: 691918
Val:   86489
Test:  86489
Total: 864896
```

Torchani API

In [7]:
```
class AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
```

```
            nn.Dropout(0.15),
            nn.Linear(256, 192),
            nn.BatchNorm1d(192),
            nn.ReLU(),
            nn.Dropout(0.15),
            nn.Linear(192, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.15),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)

net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks`b
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
```

In [8]:

```
class Bigger_AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(0.15),
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.15),
            nn.Linear(256, 128),
            nn.BatchNorm1d(128),
```

```
            nn.ReLU(),
            nn.Dropout(0.15),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)
```

In [9]:
```
class Bigger_lowerDO_AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Dropout(0.05),
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.05),
            nn.Linear(256, 128),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Dropout(0.05),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)
```

In [10]:
```
class Bigger_noDO_AtomicNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(384, 512),
            nn.BatchNorm1d(512),
            nn.ReLU(),
            nn.Linear(512, 256),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Linear(256, 128),
```

```
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.Linear(128, 1)
        )

    def forward(self, x):
        return self.layers(x)
```

In [ ]:

In [10]:
```
train_data_batch = next(iter(train_data_loader))

loss_func = nn.MSELoss()
species = train_data_batch['species'].to(device)
coords = train_data_batch['coordinates'].to(device)
true_energies = train_data_batch['energies'].to(device).float()
_, pred_energies = model((species, coords))
loss = loss_func(true_energies, pred_energies)
print(loss)
train_data_batch
```

```
tensor(2.4881, device='cuda:0', grad_fn=<MseLossBackward0>)
```

Out[10]:
```
defaultdict(list,
            {'species': tensor([[ 1,  1,  1,  ...,  0, -1, -1],
                    [ 1,  1,  1,  ..., -1, -1, -1],
                    [ 1,  2,  1,  ..., -1, -1, -1],
                    ...,
                    [ 1,  1,  1,  ...,  0, -1, -1],
                    [ 1,  1,  2,  ..., -1, -1, -1],
                    [ 3,  1,  1,  ..., -1, -1, -1]]),
             'coordinates': tensor([[[-0.9120,  1.3640,  0.0050],
                     [ 0.3138,  0.3679, -0.0376],
                     [ 0.3482, -0.9044,  0.7329],
                     ...,
                     [-0.6108, -1.2486, -1.1242],
                     [ 0.0000,  0.0000,  0.0000],
                     [ 0.0000,  0.0000,  0.0000]],

                    [[-0.0654, -0.8613, -0.0047],
                     [ 0.7938,  0.4076,  0.0034],
                     [-0.7395,  0.4685,  0.0065],
                     ...,
                     [ 0.0000,  0.0000,  0.0000],
                     [ 0.0000,  0.0000,  0.0000],
                     [ 0.0000,  0.0000,  0.0000]],

                    [[ 1.4732,  0.0194,  0.0759],
                     [ 0.1632, -0.0509, -0.5472],
                     [-0.9548, -0.7146,  0.1509],
                     ...,
                     [ 0.0000,  0.0000,  0.0000],
                     [ 0.0000,  0.0000,  0.0000],
                     [ 0.0000,  0.0000,  0.0000]],

                    ...,

                    [[-0.8867,  1.2779, -0.0021],
                     [ 0.3324,  0.3691,  0.0112],
                     [ 0.3514, -0.8869,  0.7582],
                     ...,
                     [-0.5475, -1.2799, -1.3002],
                     [ 0.0000,  0.0000,  0.0000],
                     [ 0.0000,  0.0000,  0.0000]],
```

```
                       [[ 1.3832, -0.4572, -0.0199],
                        [ 0.5448,  0.7394,  0.0201],
                        [-0.7114,  0.6814,  0.0091],
                        ...,
                        [ 0.0000,  0.0000,  0.0000],
                        [ 0.0000,  0.0000,  0.0000],
                        [ 0.0000,  0.0000,  0.0000]],

                       [[-1.5551, -0.5071, -0.0944],
                        [-0.6139,  0.6448,  0.0429],
                        [ 0.7753,  0.1398, -0.1084],
                        ...,
                        [ 0.0000,  0.0000,  0.0000],
                        [ 0.0000,  0.0000,  0.0000],
                        [ 0.0000,  0.0000,  0.0000]]]),
                'energies': tensor([-0.0073,  0.0026,  0.0173,  ..., -0.0196,  0.0056,  0.0371],
                      dtype=torch.float64)})
```

In [ ]:

In [11]:
```python
class ANITrainer:
    def __init__(self, model, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.optimizer = torch.optim.Adam(model.parameters(), learning_rate, weight_decay=l2)
        self.epoch = epoch
        self.scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(self.optimizer, mode = 'min', patience = 10, fact

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        self.model.train()

        # init data loader
        print("Initialize training data...")
        train_data_loader = train_data

        # definition of loss function: MSE is a good choice!
        loss_func = nn.MSELoss()

        # record epoch losses
```

```
        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf

        for i in tqdm(range(self.epoch), leave=True):
            self.model.train()
            train_epoch_loss = 0.0
            total_train_samples = 0

            for train_data_batch in train_data_loader:

                # compute energies
                species = train_data_batch['species'].to(device)
                coords = train_data_batch['coordinates'].to(device)
                true_E = train_data_batch['energies'].to(device).float()
                _, pred_E = self.model((species, coords))

                # compute loss
                batch_loss = loss_func(pred_E, true_E)

                # do a step
                self.optimizer.zero_grad()
                batch_loss.backward()
                torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm = 1.0)
                self.optimizer.step()


                batch_importance = len(train_data_batch['energies'])
                train_epoch_loss += batch_loss.item() * batch_importance
                total_train_samples += batch_importance

            # use the self.evaluate to get loss on the validation set

            train_epoch_loss /= total_train_samples
            self.model.eval()
            val_epoch_loss = self.evaluate(val_data)

            train_loss_list.append(train_epoch_loss)
            val_loss_list.append(val_epoch_loss)

            self.scheduler.step(val_epoch_loss)
```

```
            if early_stop:
                if val_epoch_loss < lowest_val_loss:
                    lowest_val_loss = val_epoch_loss
                    weights = self.model.state_dict()

        if draw_curve:
            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.set_yscale("log")
            # Plot train loss and validation loss
            ax.plot(range(len(train_loss_list)), train_loss_list, label='Train')
            ax.plot(range(len(val_loss_list)), val_loss_list, label='Validation')
            ax.legend()
            ax.set_xlabel("Epoch")
            ax.set_ylabel("Loss")

        if early_stop:
            self.model.load_state_dict(weights)

        return train_loss_list, val_loss_list


    def evaluate(self, data, draw_plot=False):
        self.model.eval()
        # init data loader
        data_loader = data

        # init loss function
        loss_func = nn.MSELoss()
        total_loss = 0.0
        total_samples = 0

        if draw_plot:
            true_energies_all = []
            pred_energies_all = []

        with torch.no_grad():
            for batch_data in data_loader:

                # compute energies
                species = batch_data['species'].to(device)
                coords = batch_data['coordinates'].to(device)
                true_E = batch_data['energies'].to(device).float()
```

```
                _, pred_E = self.model((species, coords))

                # compute loss
                batch_loss = loss_func(pred_E, true_E)

                batch_importance = len(batch_data['energies'])
                total_loss += batch_loss.item() * batch_importance
                total_samples += batch_importance

                if draw_plot:
                    true_energies_all.append(true_E.detach().cpu().numpy().flatten())
                    pred_energies_all.append(pred_E.detach().cpu().numpy().flatten())

        if draw_plot:
            true_energies_all = np.concatenate(true_energies_all)
            pred_energies_all = np.concatenate(pred_energies_all)
            # Report the mean absolute error
            # The unit of energies in the dataset is hartree
            # please convert it to kcal/mol when reporting the mean absolute error
            # 1 hartree = 627.5094738898777 kcal/mol
            # MAE = mean(|true - pred|)
            hartree2kcalmol = 627.5094738898777
            mae = np.mean(np.abs(true_energies_all - pred_energies_all)) *  hartree2kcalmol
            rmse = np.sqrt(np.mean((true_energies_all - pred_energies_all) ** 2)) * hartree2kcalmol
            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.scatter(true_energies_all, pred_energies_all, label=f"MAE: {mae:.2f} kcal/mol\nRMSE: {rmse:.2f} kcal/m
            ax.set_xlabel("Ground Truth")
            ax.set_ylabel("Predicted")
            xmin, xmax = ax.get_xlim()
            ymin, ymax = ax.get_ylim()
            vmin, vmax = min(xmin, ymin), max(xmax, ymax)
            ax.set_xlim(vmin, vmax)
            ax.set_ylim(vmin, vmax)
            ax.plot([vmin, vmax], [vmin, vmax], color='red')
            ax.legend()

        return total_loss / total_samples
```

Checkpoint 2 template logs the loss after each epoch, so having the x-axis as '# batch' is inaccurate. Loss for training and evaluate are divided by the total number of samples to normalize per sample size. That allows for the training and loss curves to be on the

same scale and comparable.

In [12]:
```
class lower_patience_ANITrainer:
    def __init__(self, model, learning_rate, epoch, l2):
        self.model = model

        num_params = sum(item.numel() for item in model.parameters())
        print(f"{model.__class__.__name__} - Number of parameters: {num_params}")

        self.optimizer = torch.optim.Adam(model.parameters(), learning_rate, weight_decay=l2)
        self.epoch = epoch
        self.scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(self.optimizer, mode = 'min', patience = 5, facto

    def train(self, train_data, val_data, early_stop=True, draw_curve=True):
        self.model.train()

        # init data loader
        print("Initialize training data...")
        train_data_loader = train_data

        # definition of loss function: MSE is a good choice!
        loss_func = nn.MSELoss()

        # record epoch losses
        train_loss_list = []
        val_loss_list = []
        lowest_val_loss = np.inf

        for i in tqdm(range(self.epoch), leave=True):
            self.model.train()
            train_epoch_loss = 0.0
            total_train_samples = 0

            for train_data_batch in train_data_loader:

                # compute energies
                species = train_data_batch['species'].to(device)
                coords = train_data_batch['coordinates'].to(device)
                true_E = train_data_batch['energies'].to(device).float()
                _, pred_E = self.model((species, coords))
```

```
            # compute loss
            batch_loss = loss_func(pred_E, true_E)

            # do a step
            self.optimizer.zero_grad()
            batch_loss.backward()
            torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_norm = 1.0)
            self.optimizer.step()


            batch_importance = len(train_data_batch['energies'])
            train_epoch_loss += batch_loss.item() * batch_importance
            total_train_samples += batch_importance

        # use the self.evaluate to get loss on the validation set

        train_epoch_loss /= total_train_samples
        self.model.eval()
        val_epoch_loss = self.evaluate(val_data)

        train_loss_list.append(train_epoch_loss)
        val_loss_list.append(val_epoch_loss)

        self.scheduler.step(val_epoch_loss)

        if early_stop:
            if val_epoch_loss < lowest_val_loss:
                lowest_val_loss = val_epoch_loss
                weights = self.model.state_dict()

    if draw_curve:
        fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
        ax.set_yscale("log")
        # Plot train loss and validation loss
        ax.plot(range(len(train_loss_list)), train_loss_list, label='Train')
        ax.plot(range(len(val_loss_list)), val_loss_list, label='Validation')
        ax.legend()
        ax.set_xlabel("Epoch")
        ax.set_ylabel("Loss")

    if early_stop:
        self.model.load_state_dict(weights)
```

```
    return train_loss_list, val_loss_list


def evaluate(self, data, draw_plot=False):
    self.model.eval()
    # init data loader
    data_loader = data

    # init loss function
    loss_func = nn.MSELoss()
    total_loss = 0.0
    total_samples = 0

    if draw_plot:
        true_energies_all = []
        pred_energies_all = []

    with torch.no_grad():
        for batch_data in data_loader:

            # compute energies
            species = batch_data['species'].to(device)
            coords = batch_data['coordinates'].to(device)
            true_E = batch_data['energies'].to(device).float()
            _, pred_E = self.model((species, coords))

            # compute loss
            batch_loss = loss_func(pred_E, true_E)

            batch_importance = len(batch_data['energies'])
            total_loss += batch_loss.item() * batch_importance
            total_samples += batch_importance

            if draw_plot:
                true_energies_all.append(true_E.detach().cpu().numpy().flatten())
                pred_energies_all.append(pred_E.detach().cpu().numpy().flatten())

    if draw_plot:
        true_energies_all = np.concatenate(true_energies_all)
        pred_energies_all = np.concatenate(pred_energies_all)
        # Report the mean absolute error
```

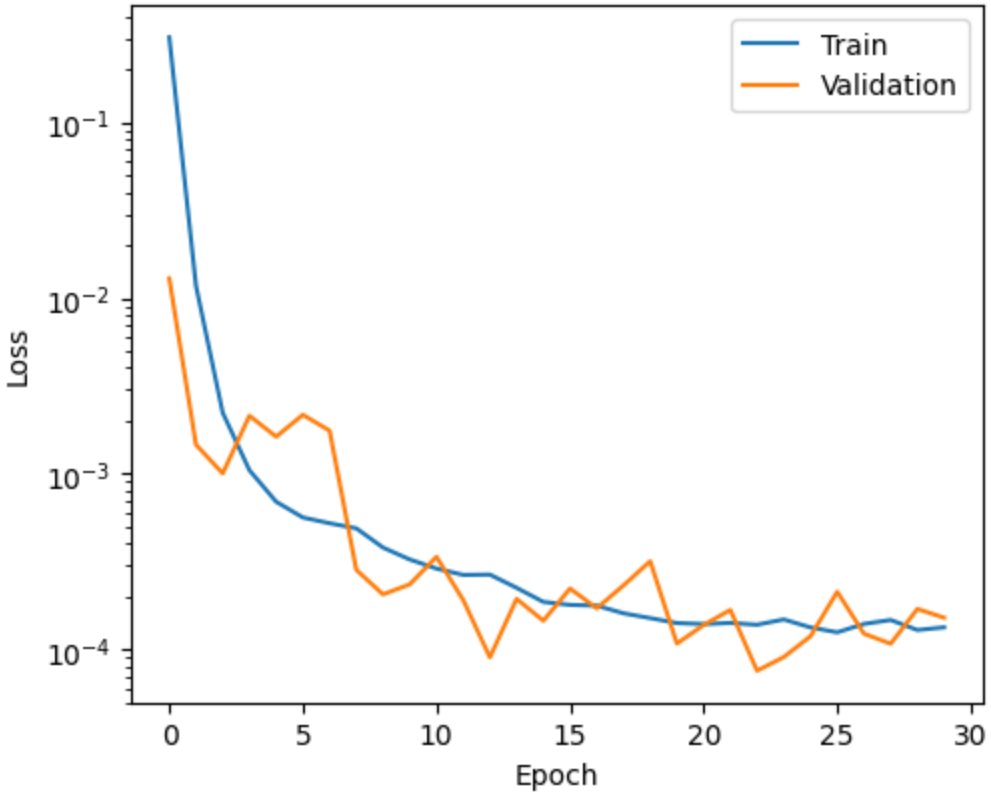
```
            # The unit of energies in the dataset is hartree
            # please convert it to kcal/mol when reporting the mean absolute error
            # 1 hartree = 627.5094738898777 kcal/mol
            # MAE = mean(|true - pred|)
            hartree2kcalmol = 627.5094738898777
            mae = np.mean(np.abs(true_energies_all - pred_energies_all)) *  hartree2kcalmol
            fig, ax = plt.subplots(1, 1, figsize=(5, 4), constrained_layout=True)
            ax.scatter(true_energies_all, pred_energies_all, label=f"MAE: {mae:.2f} kcal/mol", s=2)
            ax.set_xlabel("Ground Truth")
            ax.set_ylabel("Predicted")
            xmin, xmax = ax.get_xlim()
            ymin, ymax = ax.get_ylim()
            vmin, vmax = min(xmin, ymin), max(xmax, ymax)
            ax.set_xlim(vmin, vmax)
            ax.set_ylim(vmin, vmax)
            ax.plot([vmin, vmax], [vmin, vmax], color='red')
            ax.legend()

        return total_loss / total_samples
```

In [23]:
```
demo_set, rest = dataset.split(0.05,0.95)
train_data_demo, val_data_demo, test_data_demo = demo_set.split(0.8,0.1,0.1)
batch_size_demo = 128
train_data_loader_demo = train_data_demo.collate(batch_size_demo).cache()
val_data_loader_demo = val_data_demo.collate(batch_size_demo).cache()
test_data_loader_demo = test_data_demo.collate(batch_size_demo).cache()
```

In [24]:
```
net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
```

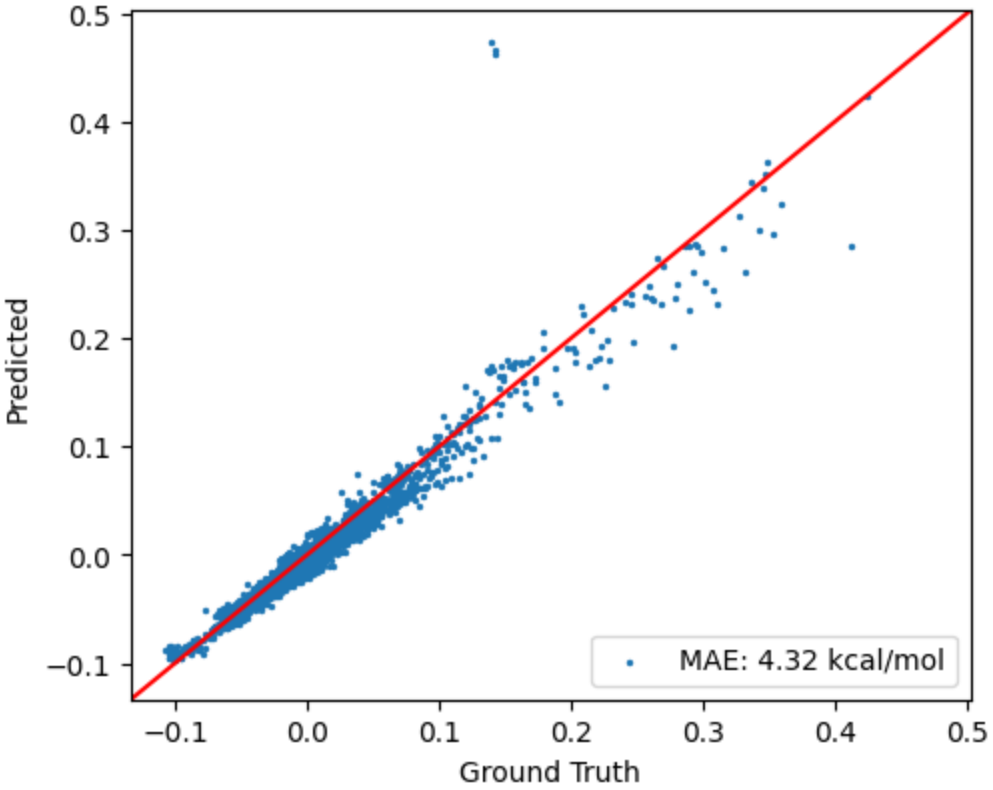
In [25]:
```
trainer = ANITrainer(model, learning_rate = 1e-3, epoch = 30, l2 = 1e-5)
train_loss, val_loss = trainer.train(train_data_loader_demo, val_data_loader_demo)
```

```
Sequential - Number of parameters: 695556
Initialize training data...
100%|██████████| 30/30 [00:54<00:00,  1.80s/it]
```



In [26]:
```
trainer.evaluate(test_data_loader_demo, draw_plot = True)
```

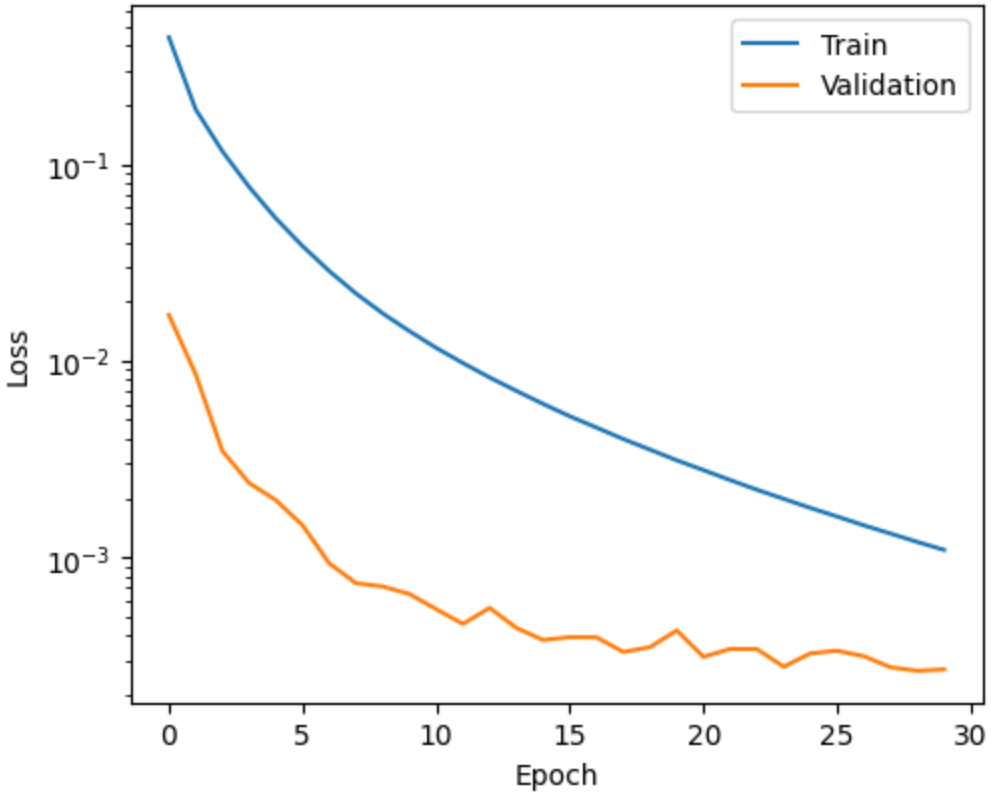
Out[26]:
```
0.00016267432094451252
```



In [27]:
```
net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata = ANITrainer(model, learning_rate = 1e-4, epoch = 30, l2 = 1e-5)
train_loss, val_loss = trainer_moredata.train(train_data_loader, val_data_loader)
```

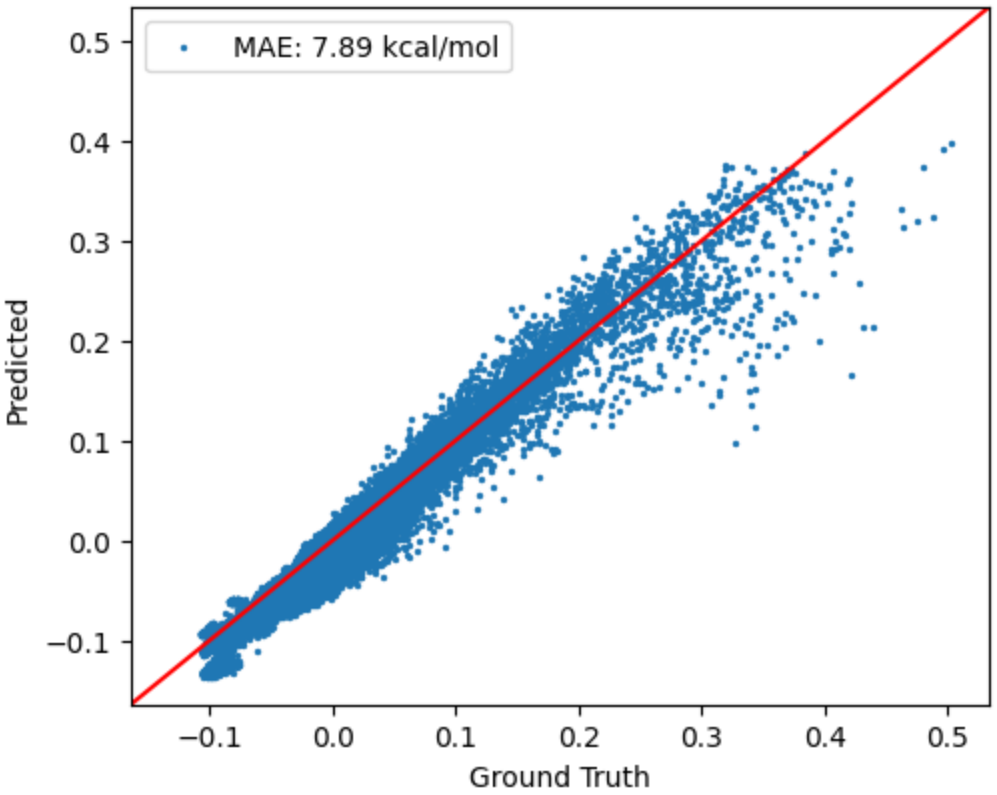
```
Sequential - Number of parameters: 695556
Initialize training data...
```

```
100%|██████████████████████████████████████████████████████████████████████████████| 30/30 [00:56<00:00,  1.88s/it]
```



In [28]:
```
trainer_moredata.evaluate(test_data_loader, draw_plot = True)
```

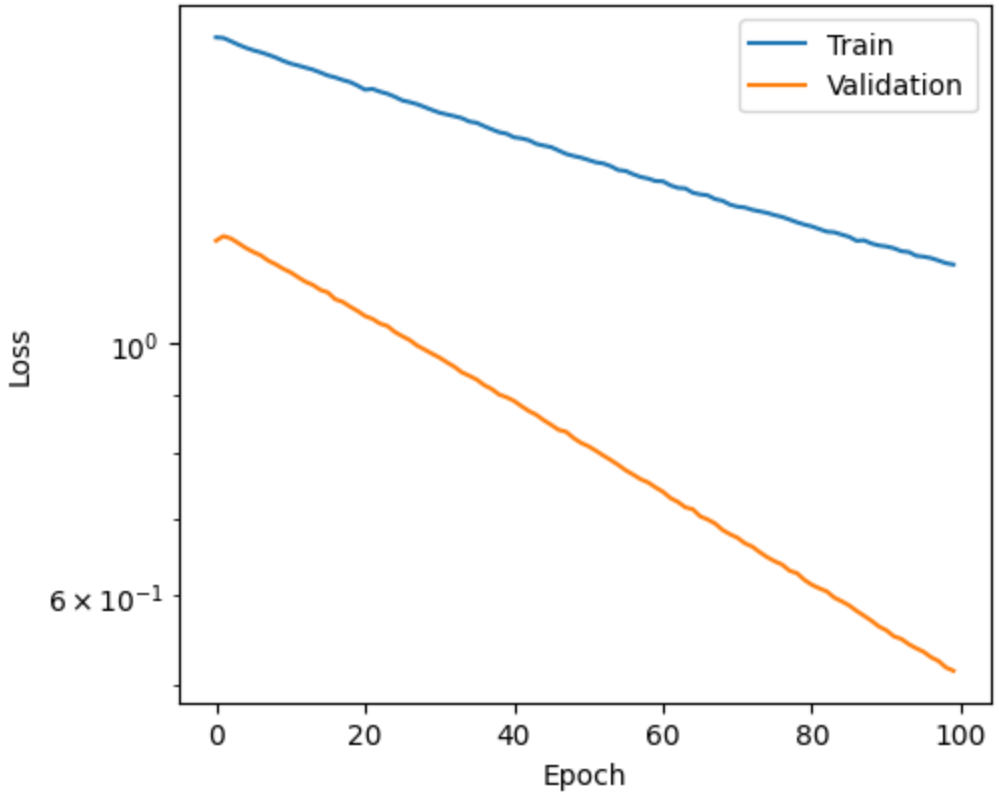
Out[28]:
```
0.0002686463669388924
```



In [29]:
```
net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_1 = ANITrainer(model, learning_rate = 1e-8, epoch = 100, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_1.train(train_data_loader, val_data_loader)
```

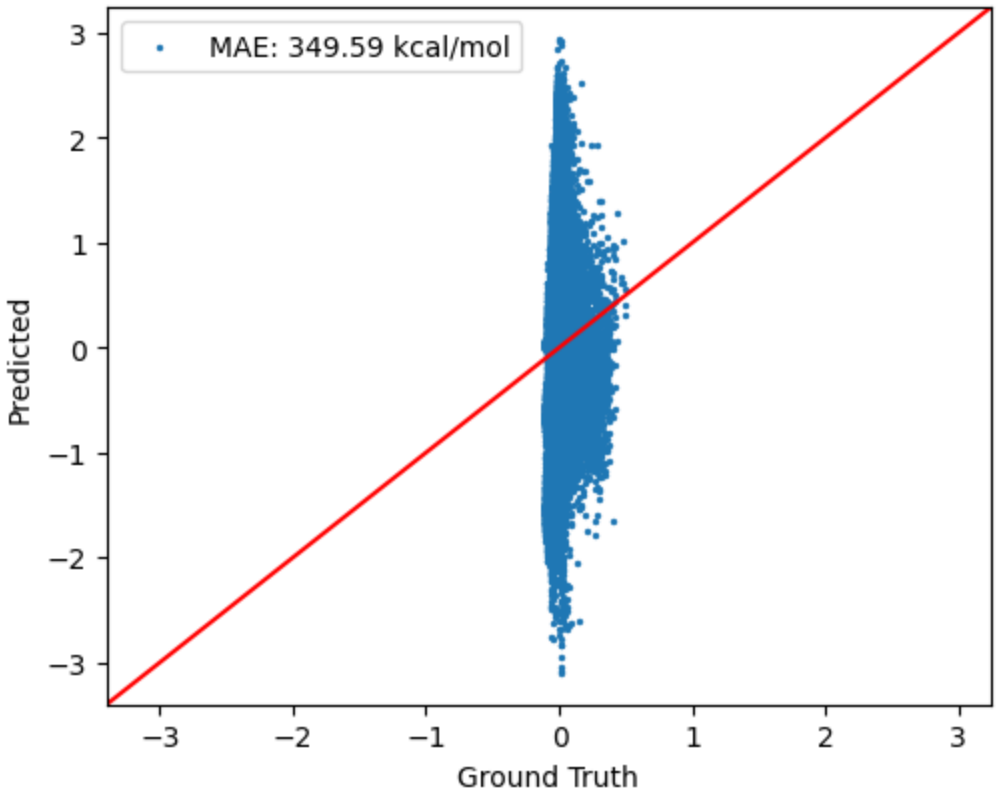
```
Sequential - Number of parameters: 695556
Initialize training data...
```

```
100%|██████████████████████████████████████████████████████████████████████████████| 100/100 [03:02<00:00,  1.83s/it]
```



In [30]:
```
trainer_moredata_1.evaluate(test_data_loader, draw_plot = True)
```

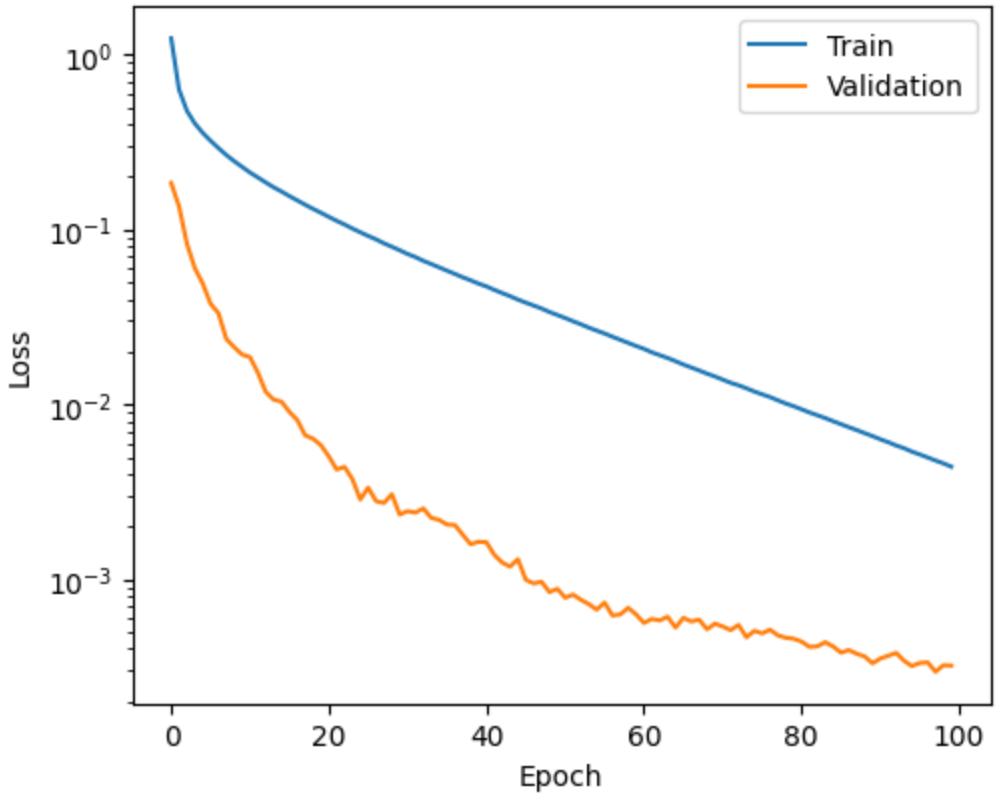
Out[30]:
```
0.5152190905448766
```



In [31]:
```
net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_2 = ANITrainer(model, learning_rate = 1e-5, epoch = 100, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_2.train(train_data_loader, val_data_loader)
```

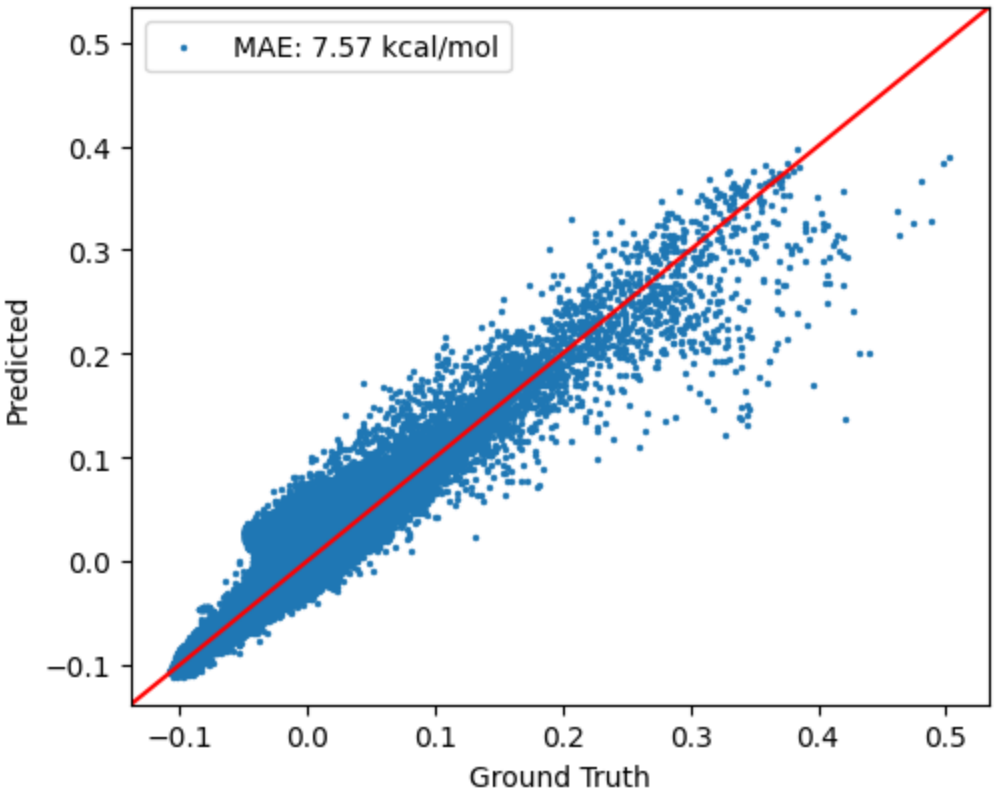
```
Sequential - Number of parameters: 695556
Initialize training data...
```

```
100%|██████████| 100/100 [03:07<00:00,  1.87s/it]
```



In [32]:
```
trainer_moredata_2.evaluate(test_data_loader, draw_plot = True)
```

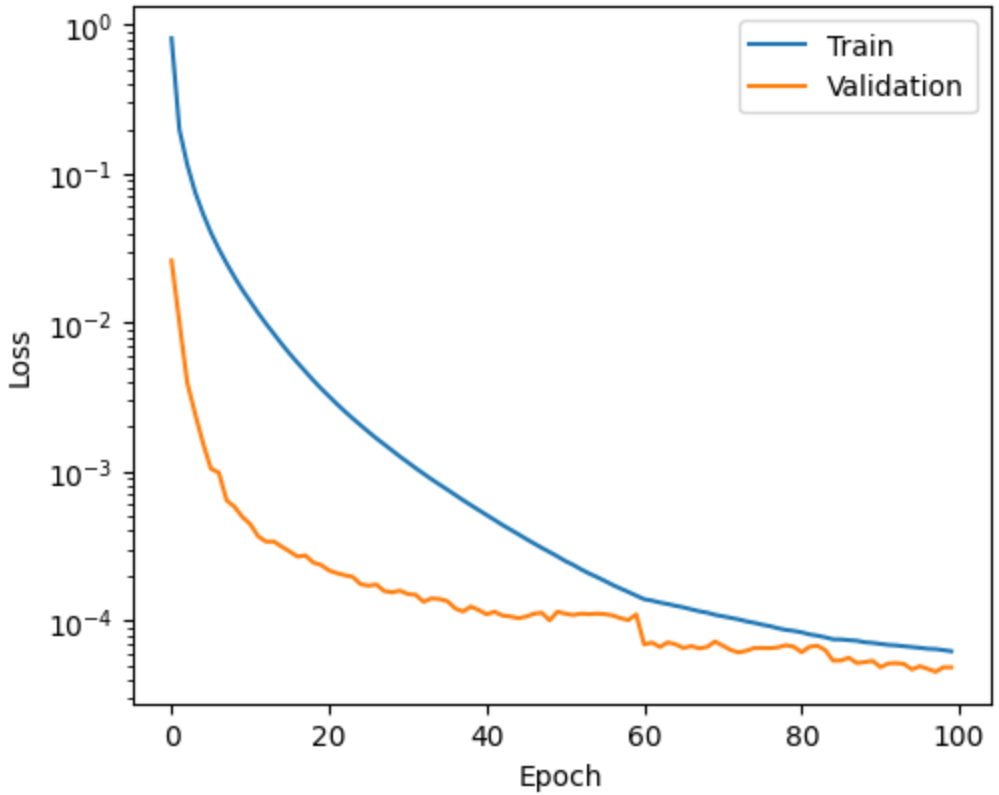
Out[32]:
```
0.0003164563611975233
```



In [33]:
```
net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_3 = ANITrainer(model, learning_rate = 1e-4, epoch = 100, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_3.train(train_data_loader, val_data_loader)
```

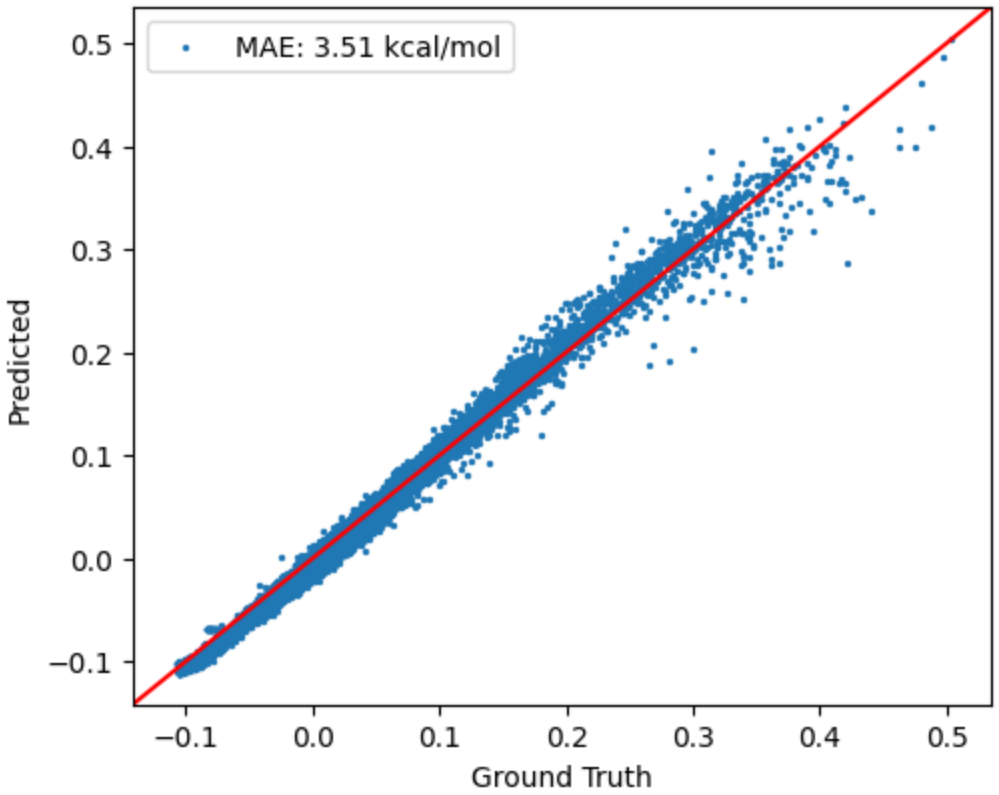
```
Sequential - Number of parameters: 695556
Initialize training data...
```

```
100%|██████████| 100/100 [03:04<00:00,  1.85s/it]
```



In [34]:
```
trainer_moredata_3.evaluate(test_data_loader, draw_plot = True)
```

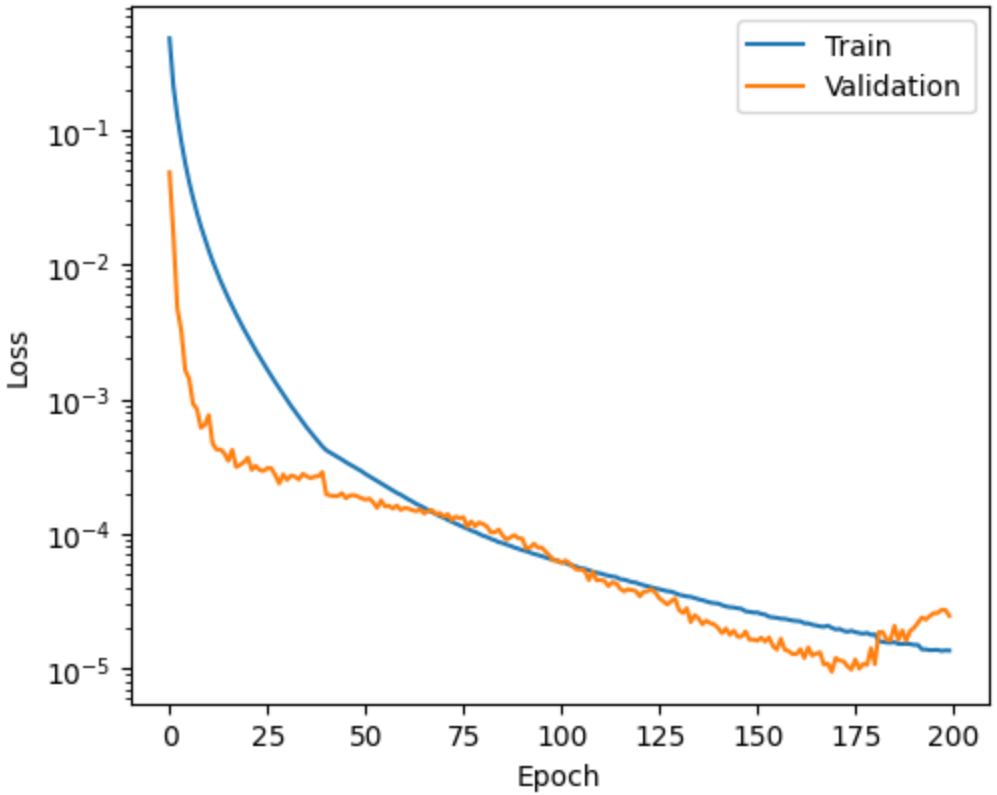
Out[34]:
```
4.89487025321561e-05
```



In [35]:
```
net_H = AtomicNet()
net_C = AtomicNet()
net_N = AtomicNet()
net_O = AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_4 = ANITrainer(model, learning_rate = 1e-4, epoch = 200, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_4.train(train_data_loader, val_data_loader)
```

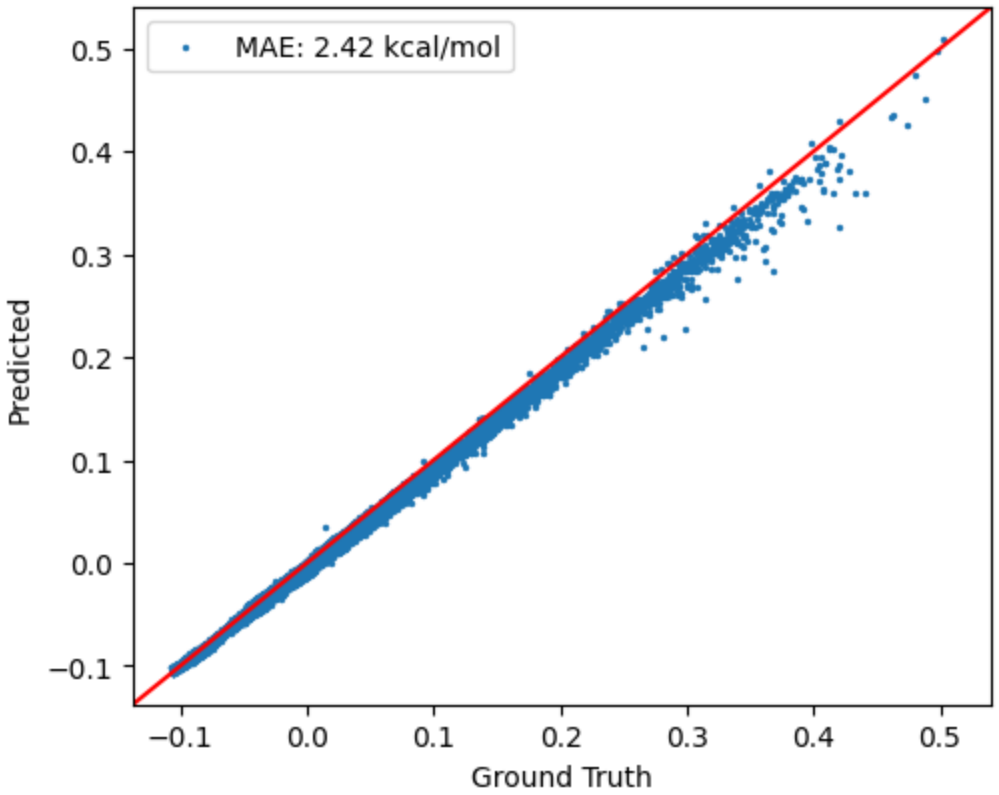
```
Sequential - Number of parameters: 695556
Initialize training data...
```

```
100%|██████████████████████████████████████████████████████████████████████████| 200/200 [06:06<00:00,  1.83s/it]
```



In [36]:
```
trainer_moredata_4.evaluate(test_data_loader, draw_plot = True)
```

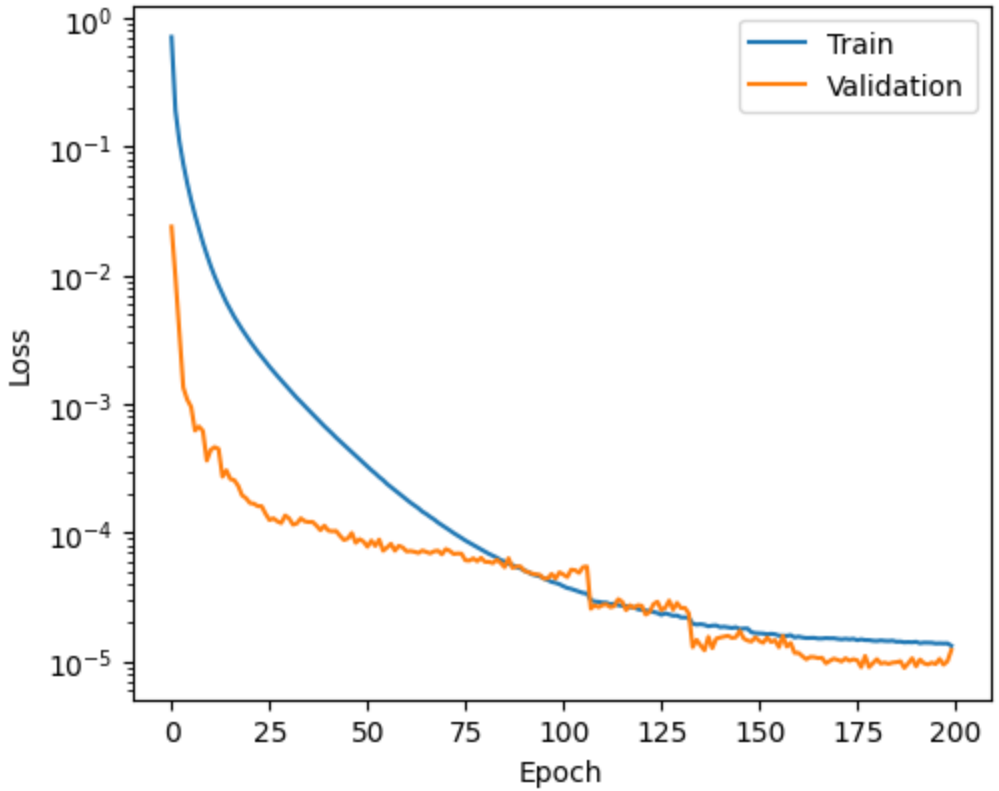
Out[36]: 2.4287488911197822e-05



In [41]:
```
net_H = Bigger_AtomicNet()
net_C = Bigger_AtomicNet()
net_N = Bigger_AtomicNet()
net_O = Bigger_AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_5 = ANITrainer(model, learning_rate = 1e-4, epoch = 200, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_5.train(train_data_loader, val_data_loader)
```

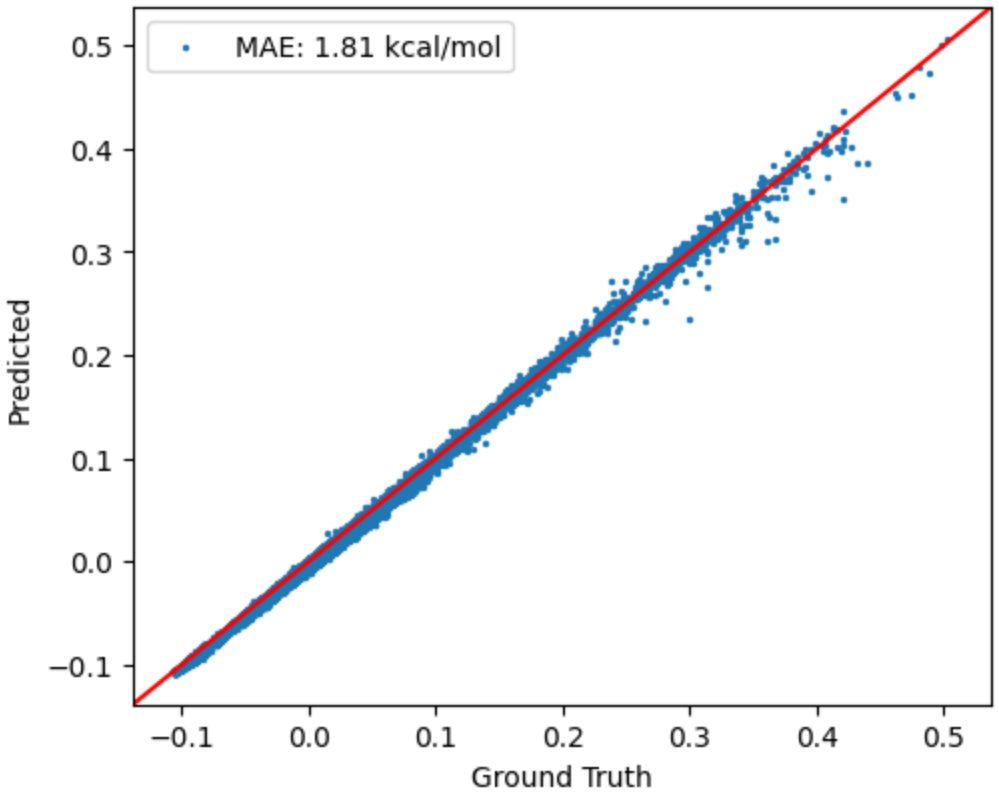
```
Sequential - Number of parameters: 1453060
Initialize training data...
```

```
100%|██████████████████████████████████████████████████████████████████████████| 200/200 [07:34<00:00,  2.27s/it]
```



In [42]:
```
trainer_moredata_5.evaluate(test_data_loader, draw_plot = True)
```

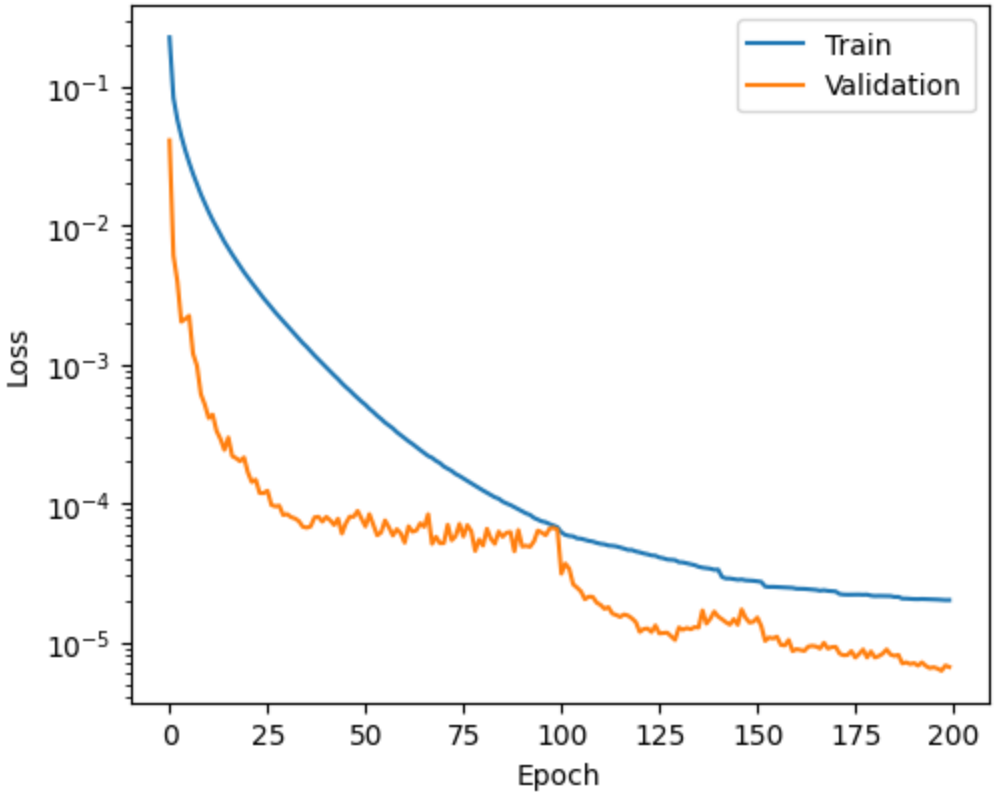
Out[42]: 1.2384741794498457e-05



In [43]:
```
net_H = Bigger_lowerDO_AtomicNet()
net_C = Bigger_lowerDO_AtomicNet()
net_N = Bigger_lowerDO_AtomicNet()
net_O = Bigger_lowerDO_AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_6 = ANITrainer(model, learning_rate = 1e-4, epoch = 200, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_6.train(train_data_loader, val_data_loader)
```

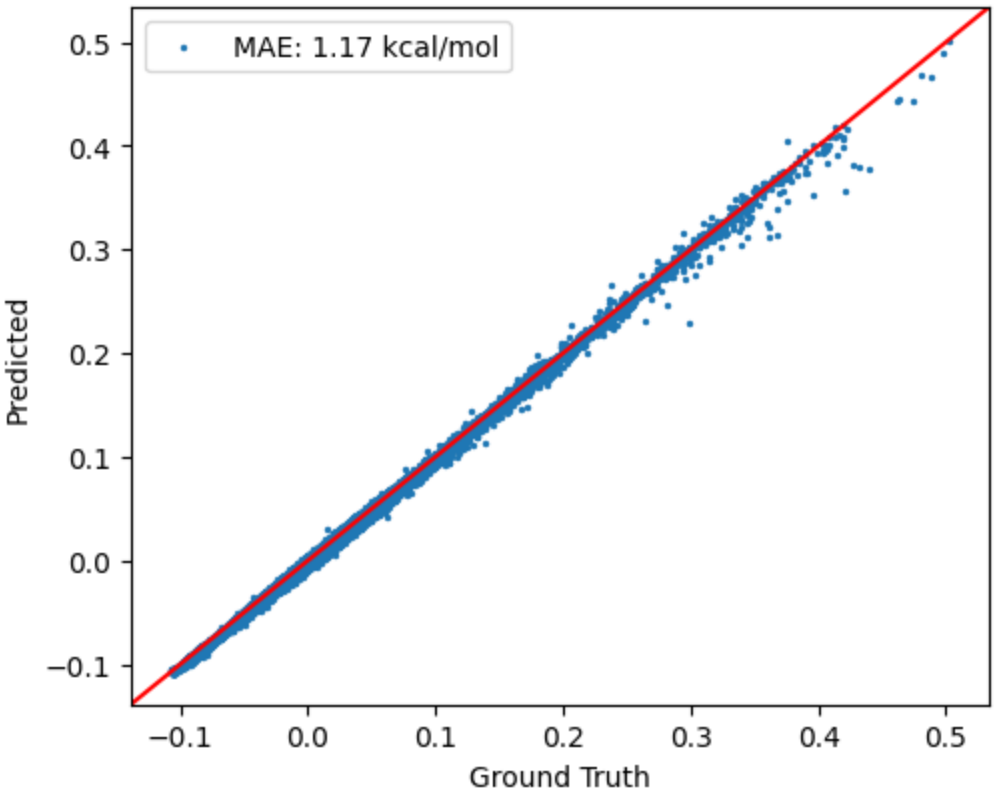
```
Sequential - Number of parameters: 1453060
Initialize training data...
```

```
100%|██████████| 200/200 [07:36<00:00,  2.28s/it]
```



In [44]:
```
trainer_moredata_6.evaluate(test_data_loader, draw_plot = True)
```

Out[44]: 6.609294922307067e-06



In [ ]:

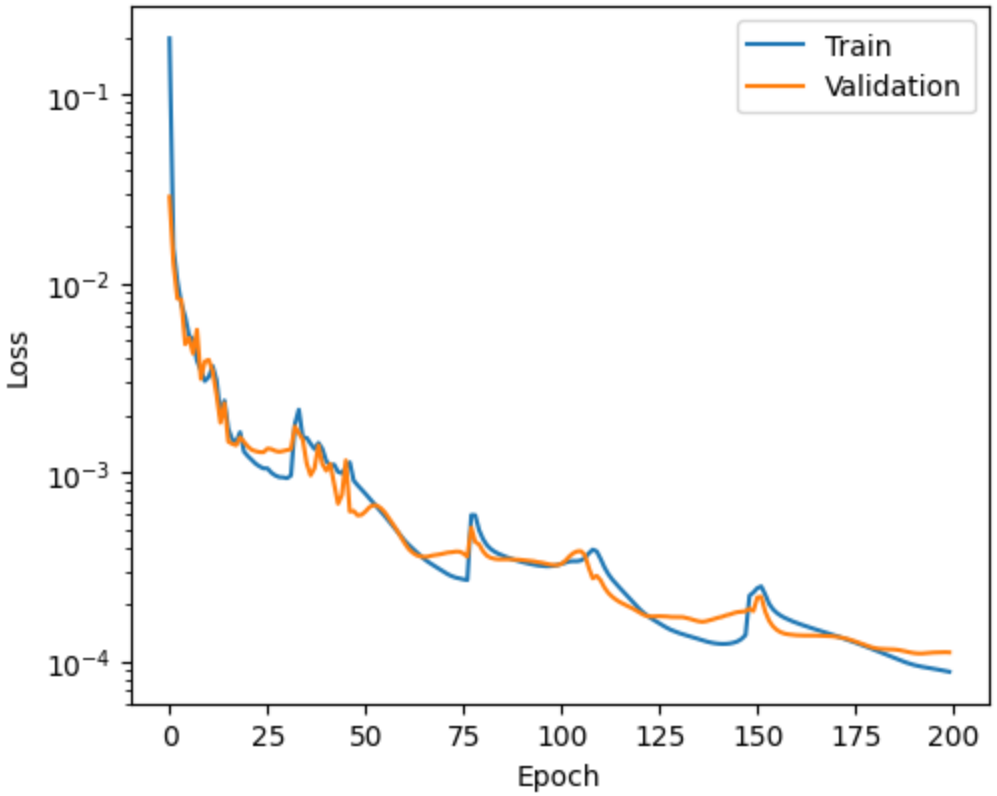
In [49]:
```
#torch.save(trainer_moredata_6.model.state_dict(), 'MH_final_proj_weights_6.pt')
```

In [ ]:

In [51]:
```
net_H = Bigger_noDO_AtomicNet()
net_C = Bigger_noDO_AtomicNet()
net_N = Bigger_noDO_AtomicNet()
net_O = Bigger_noDO_AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
```

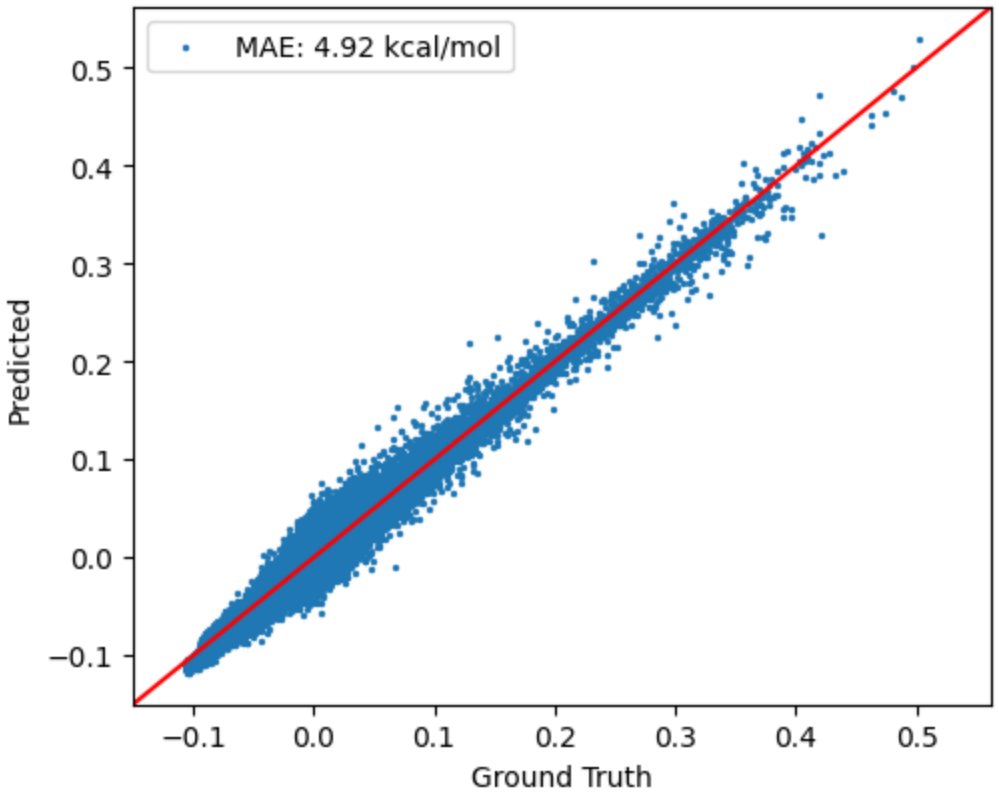
```
    ani_net
).to(device)
trainer_moredata_no_DO = ANITrainer(model, learning_rate = 1e-4, epoch = 200, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_no_DO.train(train_data_loader, val_data_loader)
```

```
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [07:10<00:00,  2.15s/it]
```



In [52]:
```
trainer_moredata_no_DO.evaluate(test_data_loader, draw_plot = True)
```

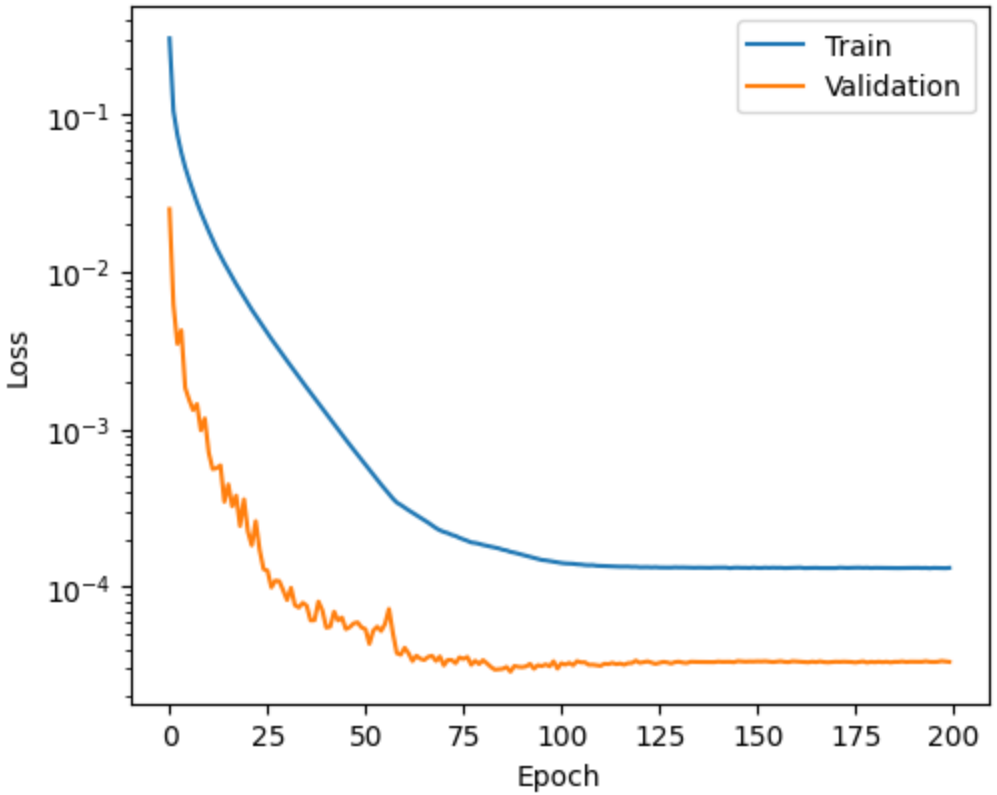
Out[52]:
```
0.00011139973513727834
```



In [ ]:

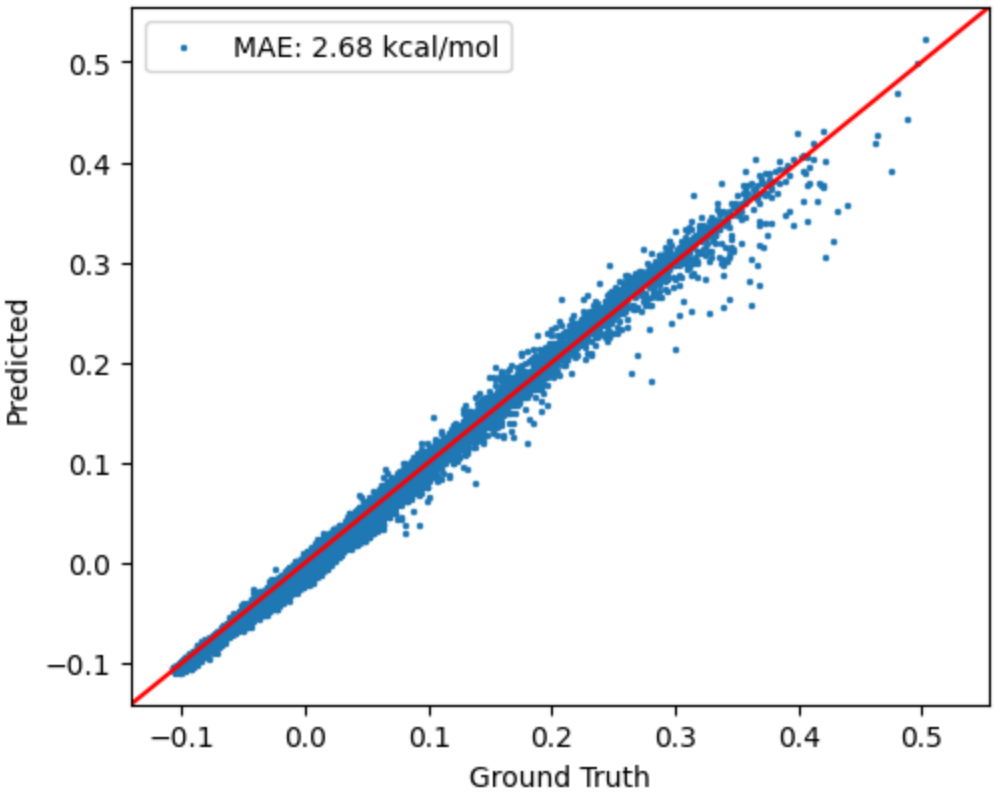
In [54]:
```
net_H = Bigger_lowerDO_AtomicNet()
net_C = Bigger_lowerDO_AtomicNet()
net_N = Bigger_lowerDO_AtomicNet()
net_O = Bigger_lowerDO_AtomicNet()

# ANI model requires a network for each atom type
# use torch.ANIModel() to compile atomic networks
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(
    aev_computer,
    ani_net
).to(device)
trainer_moredata_lower_P = lower_patience_ANITrainer(model, learning_rate = 1e-4, epoch = 200, l2 = 1e-5)
train_loss, val_loss = trainer_moredata_lower_P.train(train_data_loader, val_data_loader)
```

```
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [07:37<00:00,  2.29s/it]
```



In [55]:
```
trainer_moredata_lower_P.evaluate(test_data_loader, draw_plot = True)
```

Out[55]:
```
3.369527535516908e-05
```



In [ ]:

Checkpoint 3 write up:

I started off with 3 hidden layers with 256 nodes in the first hidden layer, 192 nodes in the second, and 128 in the third. The model achieved a test MAE of 2.42 kcal/mol. I began training with 30 epochs, a learning rate of 1e-4, and an l2 rate of 1e-5. I noticed that the MAE was 7.89 kcal/mol. Since the loss curves were still decreasing after 30 epochs, I decided to increase the number of epochs. With the same learning rate, but with 100 epochs, the MAE was 3.51 kcal/mol. At 100 epochs, I noticed that the loss curves continued to decrease. I decided to increase the number of epochs and therefore achieved a lower MAE after 200 epochs. I also experimented with smaller initial learning rates of 1e-5 and 1e-8. This turned out poorly because the models could not make any significant progress towards minimizing the loss. I could tell because the curves continued to decrease with a steep slope. Especially with a learning rate of 1e-8, the loss curves were very steep and were not able to flatten out after 100 epochs. I also put a reduce LR on plateu learning rate scheduler. This way, if the scheduler sees that there has been no improvement in loss minimization after 10

epochs, it will multiply the learning rate by 0.5. I also set a floor learning rate of 1e-8 to prevent having the learning rate so small that there is no meaningful progress towards optimization. I began with a l2 hyperparameter of 1e-5. L2 regression penalizes larger weights. Increasing the l2 hyperparameter would help if the model is overfitting. However, since the loss curves continue to decrease quite nicely, increasing the hyperparameter could hurt the result. I also decided to use the default batch size given from checkpoint 1. The loss curves do not look like they are fluctuating and the the loss continues to decrease. The data loader also loads the correct batch size of data into the model training, so no need for a hyperparameter when I initiate ANITrainer.

I then decided to increase the size of the hidden layers to 512 nodes in the first hidden layer, 256 nodes in the second, and 128 in the third. As a note, both architectures had 384 nodes as an input at 1 node as output. With a larger capacity, the MAE dropped to 1.81 kcal/mol. I then wanted to see if I could drop the MAE even lower by lowering drop out. I lowered drop out from 0.15 to 0.05. The result was 1.17 kcal/mol. For both cuves at 200 epochs, they began to flatten out. I even tried removing drop out all together to get it even lower. However, that resulted in an MAE of 4.92 kcal/mol. With the same parameters as the model that produced the 1.17 kcal/mol, I wanted to lower the patience of the learning rate scheduler from 10 to 5. This will allow for more frequent updates when the during later epochs when the loss curve is mostly flattened out. However, we see that the loss curve flattened out so much after 100 epochs. This tells us that the learning rate got so small that any progress towards minimizing loss was not meaningful. The MAE turned out to be 2.68 kcal/mol. Although I was trying to get MAE to be below 1, as that is the typical allowed error for what is considered good for a realistic chemical prediction, 1.17 kcal/mol was the closest I got so far. It was trained on a learning rate of 1e-4, 200 epochs, and an l2 rate of 1e-5.

In [13]:
```
def traditional_torchani_kfold_cv(
    k,
    full_dataset,
    model_builder,
    epochs,
    batch_size=8192,
    learning_rate=1e-4,
    l2=1e-5,
    draw_curve=True,
    verbose=True,
):

    """
    K-fold CV for torchani models.

    Parameters
    ----------
```

```
    k : int
        Number of folds.
    full_dataset : torchani TransformableIterable
    model_builder : callable
        Returns a fresh model each fold
    epochs, batch_size, learning_rate, l2 : training hyperparameters
    """
    if draw_curve:
        n_row, n_col = int(np.ceil(k / 3)), 3
        fig, axes = plt.subplots(n_row, n_col, figsize=(5*n_col, 4*n_row), constrained_layout=True)
        axes = axes.flatten()

    train_mse_list, test_mse_list = [], []
    test_mae_list, test_rmse_list = [], []
    trainers = []

    shuffled = full_dataset.shuffle()
    fractions = [1.0 / k] * k
    folds = shuffled.split(*fractions)

    for i in range(k):
        if verbose:
            print(f"\n Fold {i+1}/{k} ")

        test_fold = folds[i]
        val_idx = (i + 1) % k
        val_fold = folds[val_idx]
        train_fold_list = [folds[j] for j in range(k) if j != i and j != val_idx]

        train_batches = []
        for f in train_fold_list:
            for batch in f.collate(batch_size).cache():
                train_batches.append(batch)

        val_loader = val_fold.collate(batch_size).cache()
        test_loader = test_fold.collate(batch_size).cache()

        model = model_builder()
        trainer = ANITrainer(model, learning_rate=learning_rate, epoch=epochs, l2=l2)

        train_losses, val_losses = trainer.train(
            train_batches, val_loader,
```

```
            early_stop=True, draw_curve=False
        )

        test_mse = trainer.evaluate(test_loader, draw_plot=False)
        train_mse_list.append(trainer.evaluate(train_batches, draw_plot=False))
        test_mse_list.append(test_mse)

        hartree2kcalmol = 627.5094738898777
        model.eval()
        true_all, pred_all = [], []
        with torch.no_grad():
            for batch in test_loader:
                species = batch['species'].to(device)
                coords = batch['coordinates'].to(device)
                true_E = batch['energies'].to(device).float()
                _, pred_E = model((species, coords))
                true_all.append(true_E.cpu().numpy().flatten())
                pred_all.append(pred_E.cpu().numpy().flatten())

        true_all = np.concatenate(true_all)
        pred_all = np.concatenate(pred_all)
        mae = np.mean(np.abs(true_all - pred_all)) * hartree2kcalmol
        rmse = np.sqrt(np.mean((true_all - pred_all) ** 2)) * hartree2kcalmol
        test_mae_list.append(mae)
        test_rmse_list.append(rmse)

        trainers.append(trainer)

        if draw_curve:
            axes[i].set_yscale("log")
            axes[i].plot(range(len(train_losses)), train_losses, label='Train')
            axes[i].plot(range(len(val_losses)), val_losses, label='Validation')
            axes[i].set_xlabel('Epoch')
            axes[i].set_ylabel('Loss')
            axes[i].set_title(f"Fold {i+1}")
            axes[i].legend()

        if verbose:
            print(f"Fold {i+1} — MAE: {mae:.3f} kcal/mol, RMSE: {rmse:.3f} kcal/mol")

    if draw_curve:
        for j in range(k, len(axes)):
```

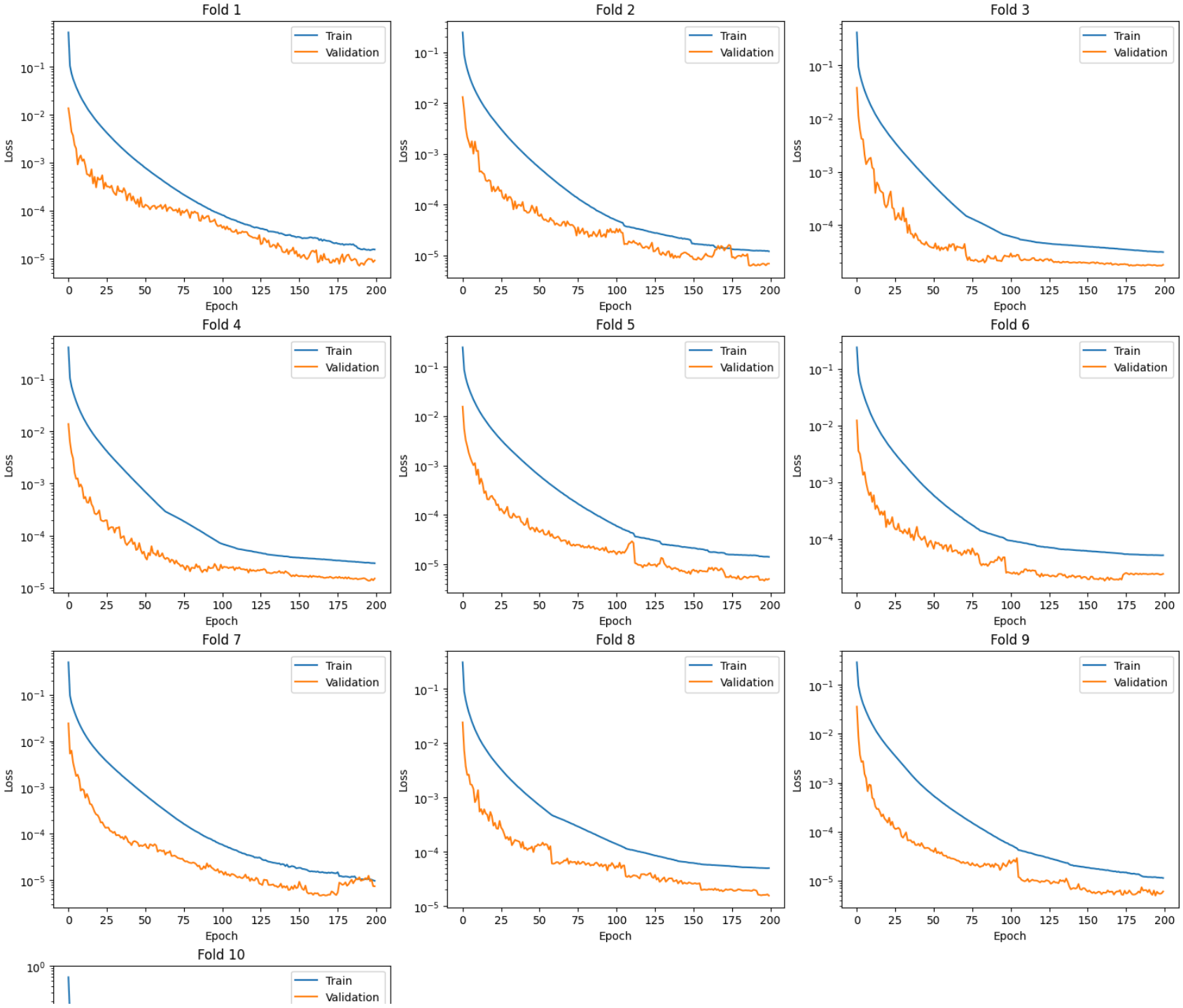
```
            axes[j].axis('off')

    if verbose:
        print(f"\n Final K-fold results")
        print(f"Test MAE:  {np.mean(test_mae_list):.3f} +/- {np.std(test_mae_list):.3f} kcal/mol")
        print(f"Test RMSE: {np.mean(test_rmse_list):.3f} +/- {np.std(test_rmse_list):.3f} kcal/mol")
        print(f"Test MSE:  {np.mean(test_mse_list):.3e} +/- {np.std(test_mse_list):.3e}")
        print(f"\nPer-fold MAE:  {[f'{m:.3f}' for m in test_mae_list]}")
        print(f"Per-fold RMSE: {[f'{r:.3f}' for r in test_rmse_list]}")

    return trainers, test_mae_list, test_rmse_list
```

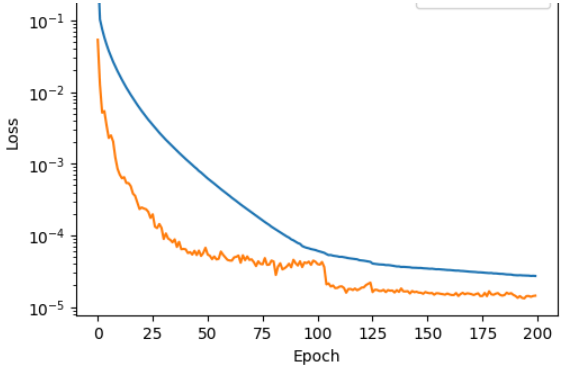
In [14]:
```
def make_best_model():
    nets = [Bigger_lowerDO_AtomicNet() for _ in range(4)]
    ani_net = torchani.ANIModel(nets)
    return nn.Sequential(aev_computer, ani_net).to(device)

trainers_2, train_errs_2, test_errs_2 = traditional_torchani_kfold_cv(
    k=10,
    full_dataset=dataset,
    model_builder=make_best_model,
    epochs=200,
    batch_size=8192,
    learning_rate=1e-4,
    l2=1e-5,
)
```

```
 Fold 1/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [10:20<00:00,  3.10s/it]
Fold 1 — MAE: 1.477 kcal/mol, RMSE: 1.893 kcal/mol

 Fold 2/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:48<00:00,  2.94s/it]
```

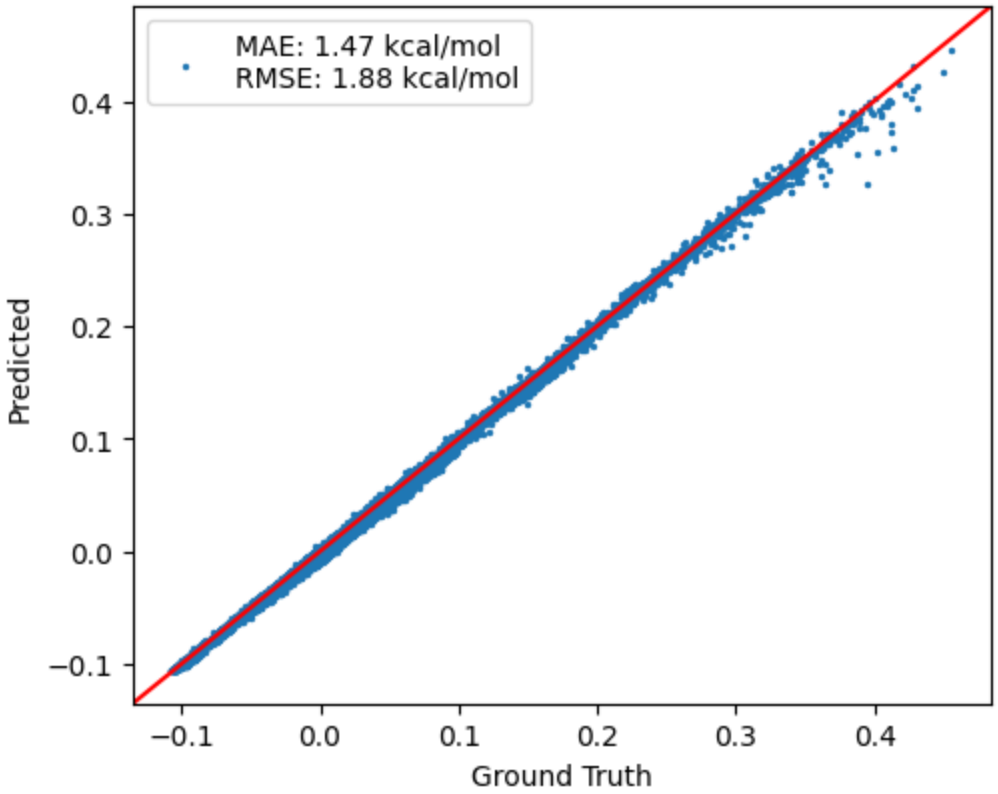
```
Fold 2 — MAE: 1.329 kcal/mol, RMSE: 1.656 kcal/mol

 Fold 3/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:42<00:00,  2.91s/it]
Fold 3 — MAE: 2.132 kcal/mol, RMSE: 2.647 kcal/mol

 Fold 4/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:43<00:00,  2.92s/it]
Fold 4 — MAE: 1.929 kcal/mol, RMSE: 2.443 kcal/mol

 Fold 5/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:45<00:00,  2.93s/it]
Fold 5 — MAE: 1.016 kcal/mol, RMSE: 1.393 kcal/mol

 Fold 6/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:44<00:00,  2.92s/it]
Fold 6 — MAE: 2.620 kcal/mol, RMSE: 3.087 kcal/mol

 Fold 7/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:55<00:00,  2.98s/it]
Fold 7 — MAE: 1.181 kcal/mol, RMSE: 1.676 kcal/mol

 Fold 8/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:50<00:00,  2.95s/it]
```

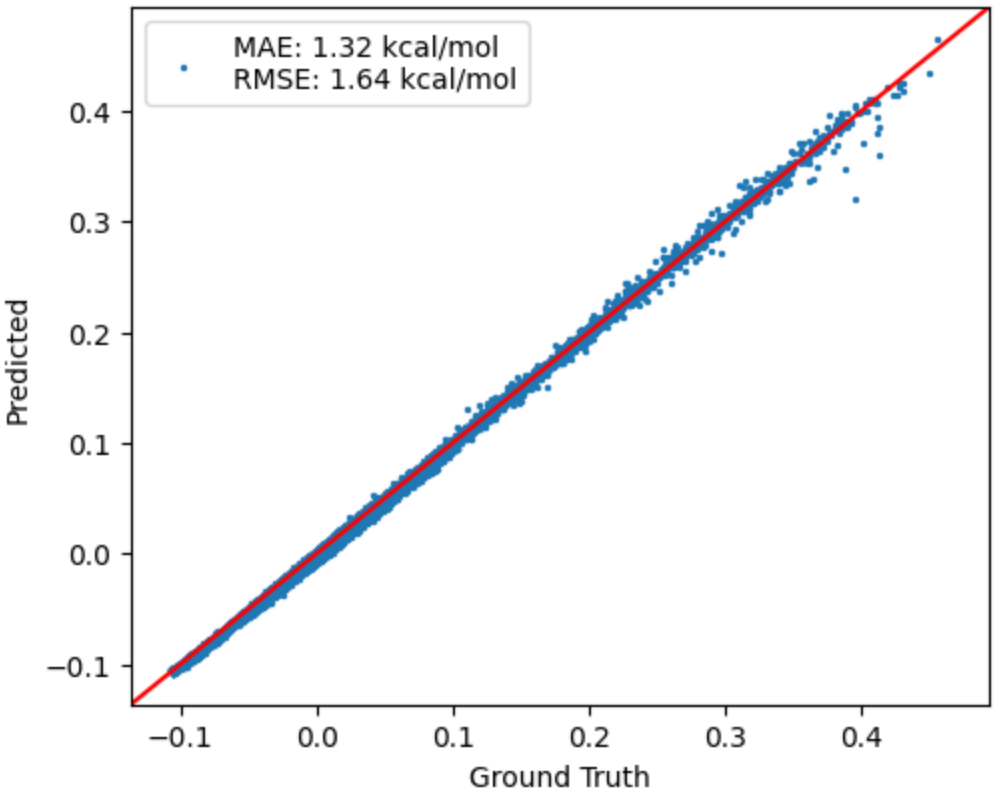
```
Fold 8 — MAE: 2.011 kcal/mol, RMSE: 2.522 kcal/mol

 Fold 9/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:48<00:00,  2.94s/it]
Fold 9 — MAE: 1.078 kcal/mol, RMSE: 1.519 kcal/mol

 Fold 10/10
Sequential - Number of parameters: 1453060
Initialize training data...
100%|██████████| 200/200 [09:46<00:00,  2.93s/it]
Fold 10 — MAE: 1.973 kcal/mol, RMSE: 2.388 kcal/mol

 Final K-fold results
Test MAE:  1.675 +/- 0.506 kcal/mol
Test RMSE: 2.123 +/- 0.539 kcal/mol
Test MSE:  1.218e-05 +/- 6.007e-06

Per-fold MAE:  ['1.477', '1.329', '2.132', '1.929', '1.016', '2.620', '1.181', '2.011', '1.078', '1.973']
Per-fold RMSE: ['1.893', '1.656', '2.647', '2.443', '1.393', '3.087', '1.676', '2.522', '1.519', '2.388']
```

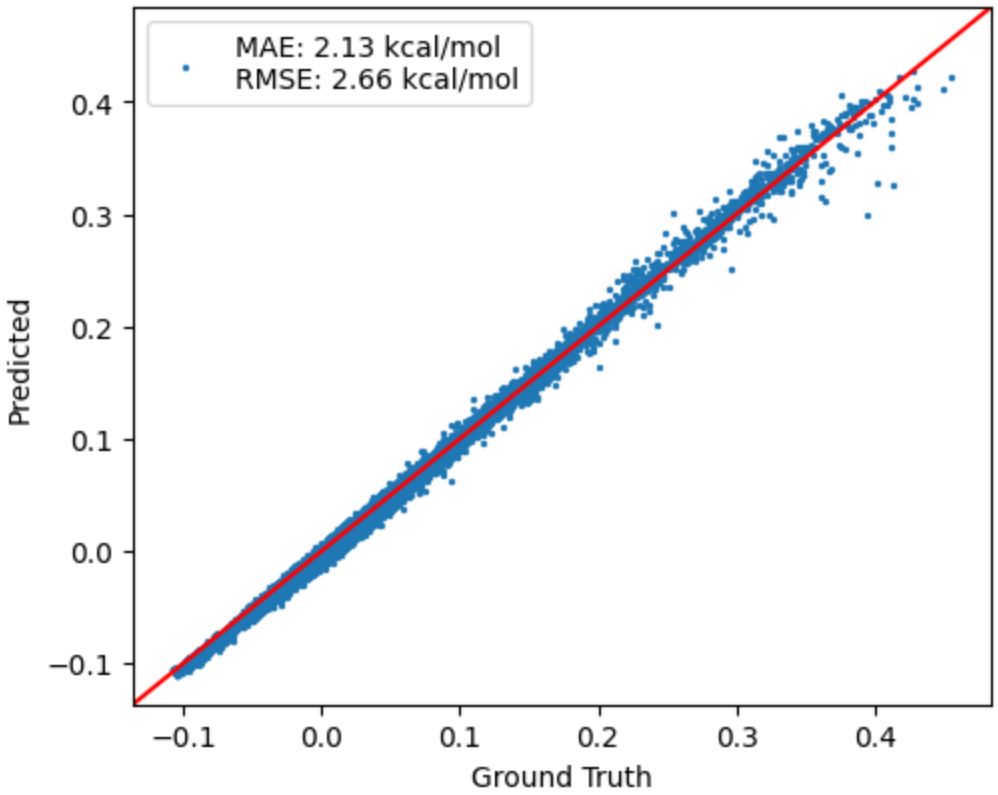

Fold 1
Fold 2
Fold 3
Fold 4
Fold 5
Fold 6
Fold 7
Fold 8
Fold 9
Fold 10
Train
Validation
Loss
Epoch

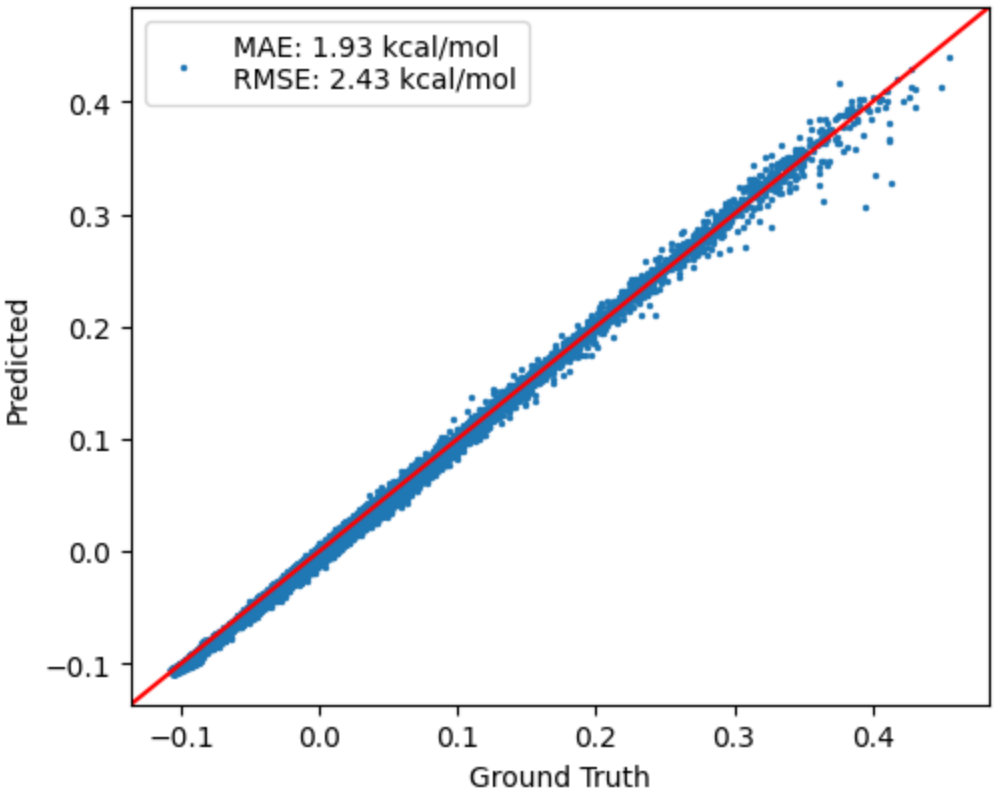


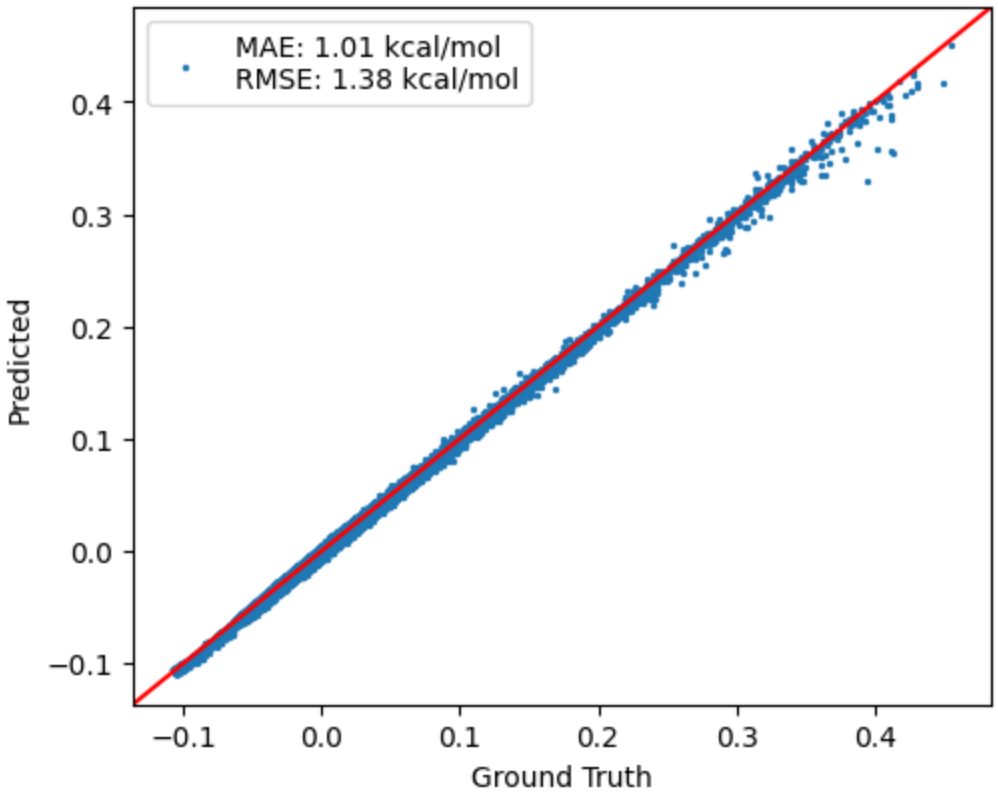
In [15]:
```
for trainer_kfold in trainers_2:
    trainer_kfold.evaluate(test_data_loader, draw_plot=True)
```

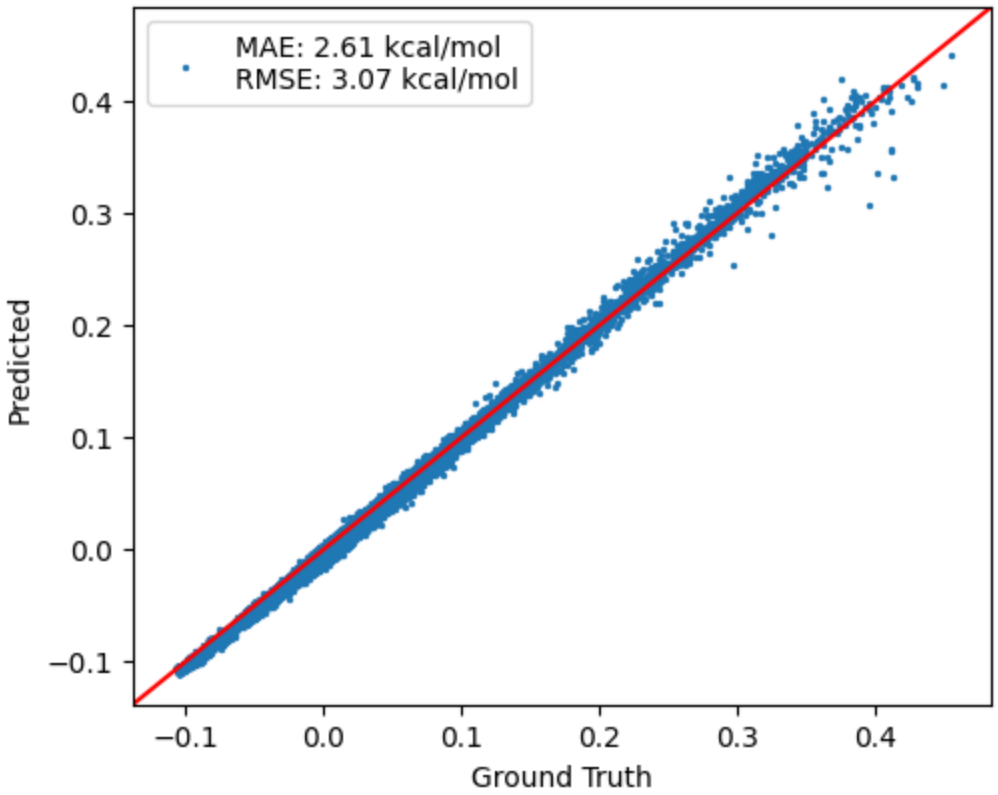


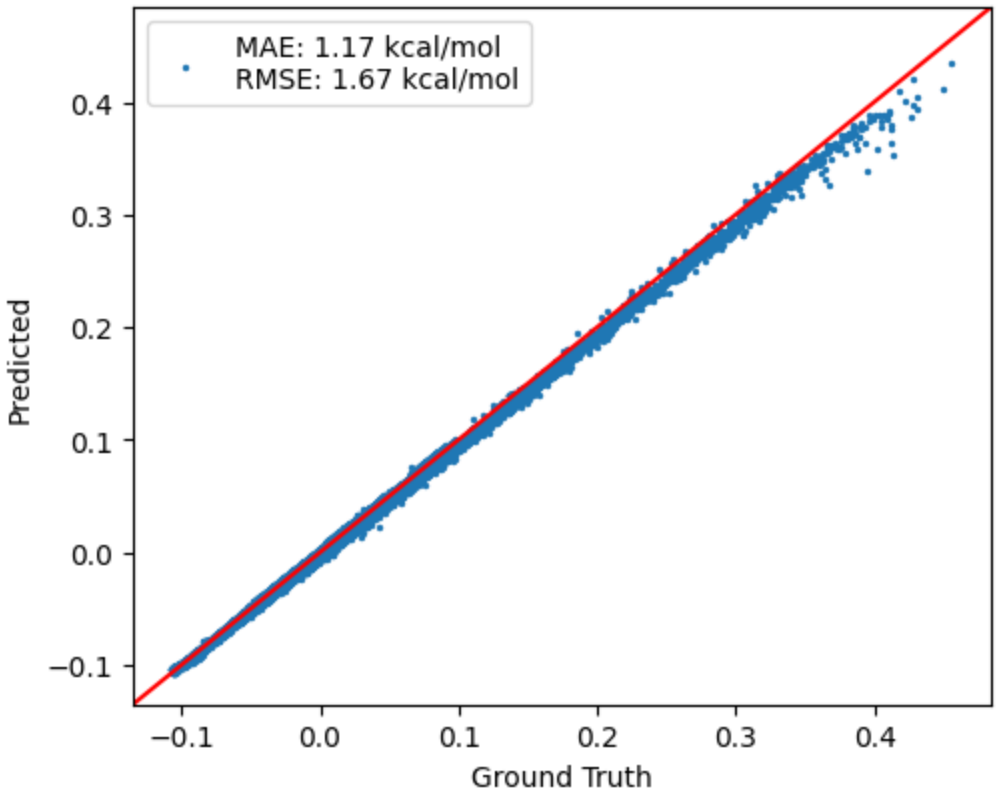

MAE: 1.32 kcal/mol
RMSE: 1.64 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4

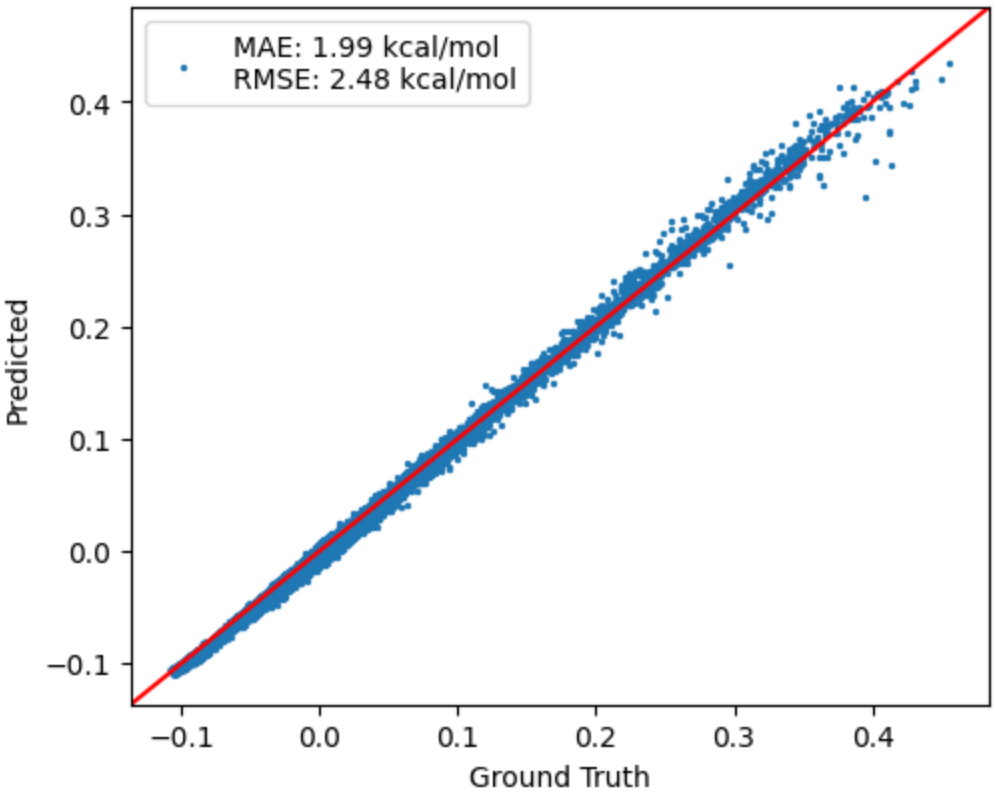

MAE: 2.13 kcal/mol
RMSE: 2.66 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4

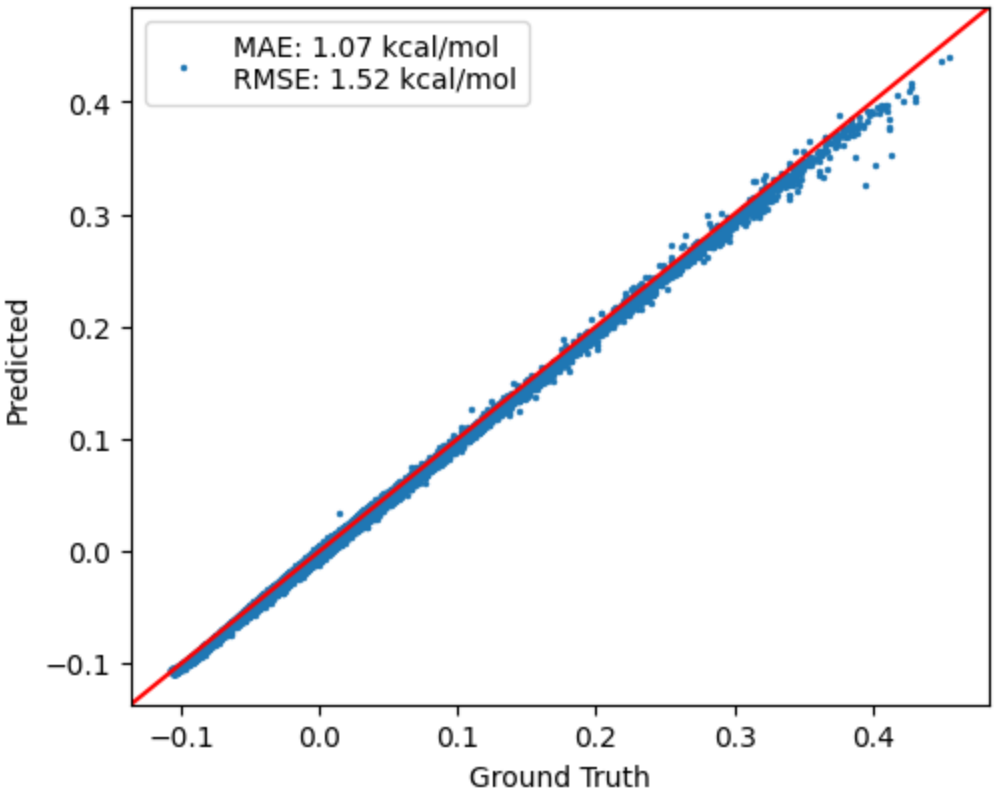

MAE: 1.93 kcal/mol
RMSE: 2.43 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4

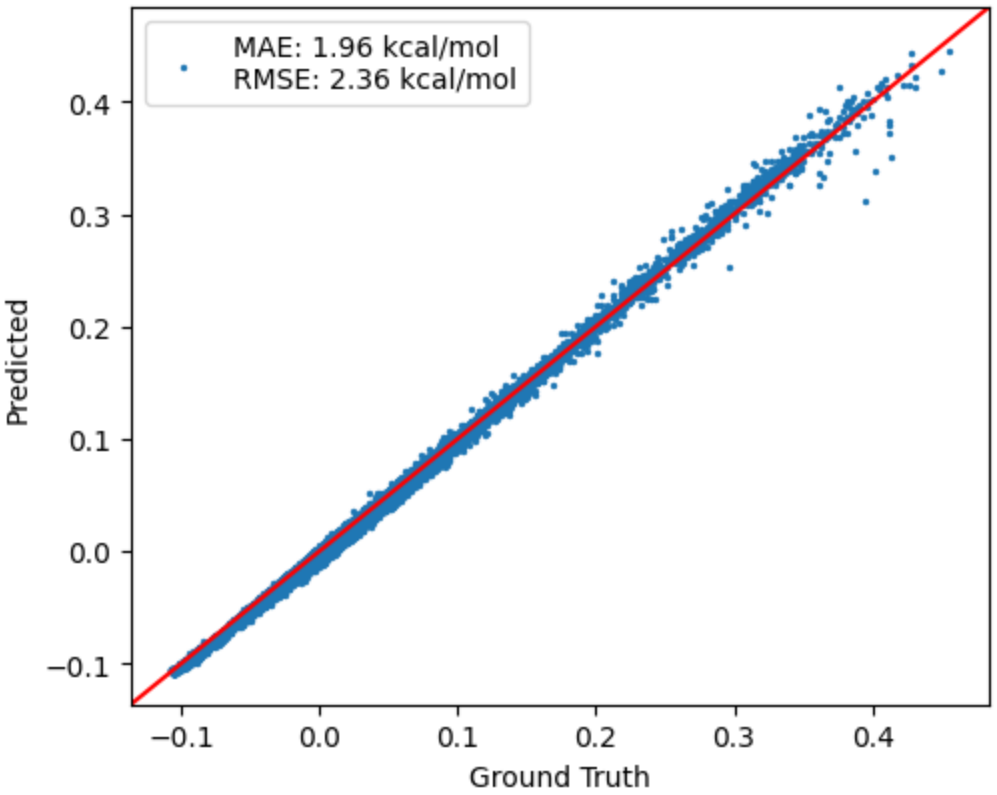

MAE: 1.01 kcal/mol
RMSE: 1.38 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4


MAE: 2.61 kcal/mol
RMSE: 3.07 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4


MAE: 1.17 kcal/mol
RMSE: 1.67 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4


MAE: 1.99 kcal/mol
RMSE: 2.48 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4


MAE: 1.07 kcal/mol
RMSE: 1.52 kcal/mol
Predicted
Ground Truth
0.4
0.3
0.2
0.1
0.0
−0.1
−0.1
0.0
0.1
0.2
0.3
0.4



In [16]:
```
#torch.save(trainers_2[4].model.state_dict(), 'best_model_fold5_MAE101.pt')
```

In [17]:
```
net_H = Bigger_lowerDO_AtomicNet()
net_C = Bigger_lowerDO_AtomicNet()
net_N = Bigger_lowerDO_AtomicNet()
net_O = Bigger_lowerDO_AtomicNet()
ani_net = torchani.ANIModel([net_H, net_C, net_N, net_O])
model = nn.Sequential(aev_computer, ani_net).to(device)
model.load_state_dict(torch.load('best_model_fold5_MAE101.pt', map_location=device, weights_only=True))
trainer_reload = ANITrainer(model, learning_rate=1e-4, epoch=200, l2=1e-5)
```

```
Sequential - Number of parameters: 1453060
```

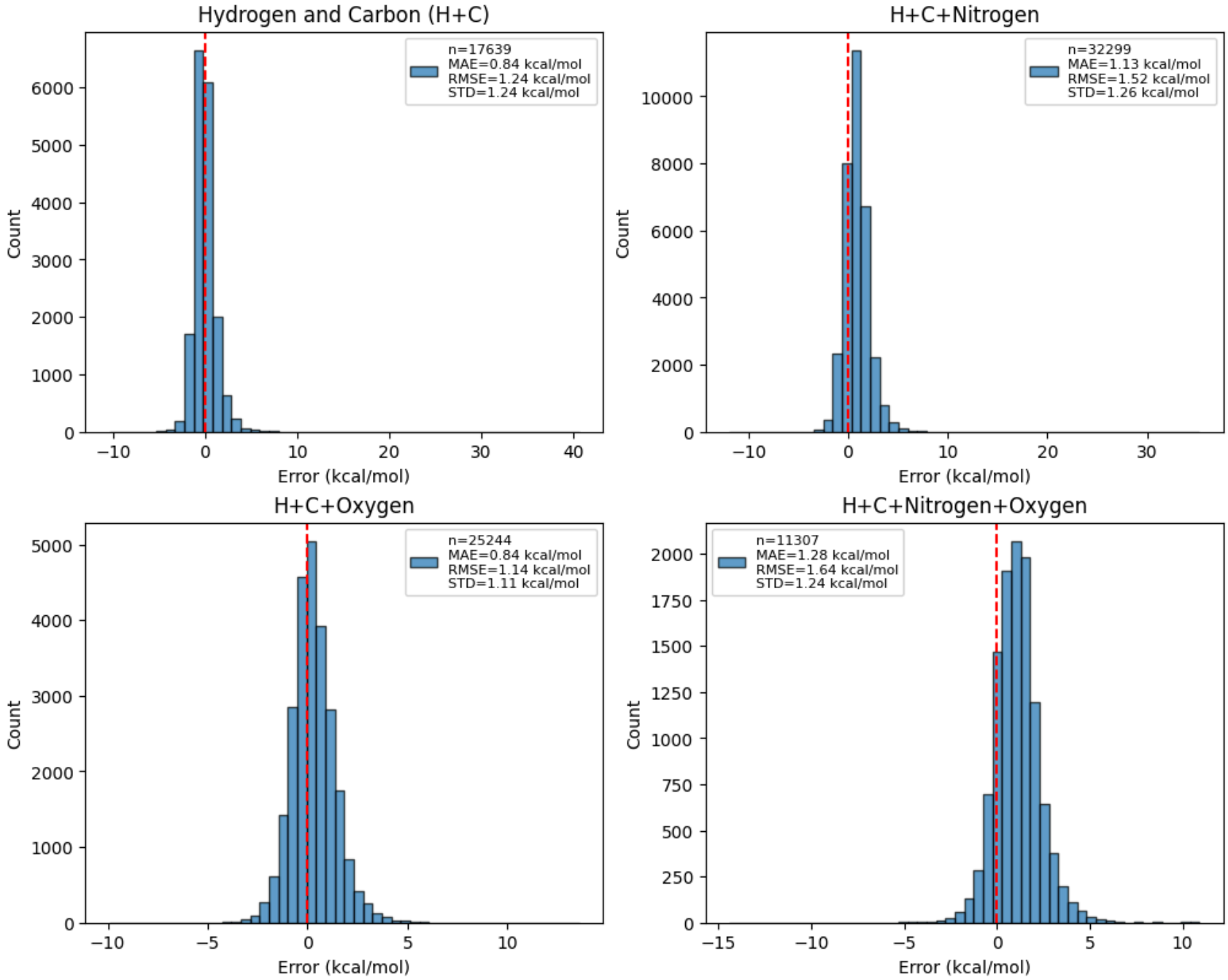
In [18]:
```
model.eval()
hartree2kcalmol = 627.5094738898777
all_true, all_pred, all_has_N, all_has_O = [], [], [], []
with torch.no_grad():
```

```
    for batch in test_data_loader:
        species = batch['species'].to(device)
        coords  = batch['coordinates'].to(device)
        true_E  = batch['energies'].to(device).float()
        _, pred_E = model((species, coords))
        species_np = species.cpu().numpy()
        true_np    = true_E.cpu().numpy().flatten()
        pred_np    = pred_E.cpu().numpy().flatten()
        for i, mol in enumerate(species_np):
            mol = mol[mol >= 0]
            all_true.append(true_np[i])
            all_pred.append(pred_np[i])
            all_has_N.append(2 in mol)
            all_has_O.append(3 in mol)
all_true  = np.array(all_true)
all_pred  = np.array(all_pred)
all_has_N = np.array(all_has_N)
all_has_O = np.array(all_has_O)
errors    = (all_true - all_pred) * hartree2kcalmol
categories = {
    'Hydrogen and Carbon (H+C)': ~all_has_N & ~all_has_O,
    'H+C+Nitrogen':    all_has_N & ~all_has_O,
    'H+C+Oxygen':    ~all_has_N &  all_has_O,
    'H+C+Nitrogen+Oxygen':   all_has_N &  all_has_O,
}
print(f"{'Category':<30} {'Count':>6} {'MAE':>10} {'RMSE':>10}")
print("-" * 58)
for name, mask in categories.items():
    if mask.any():
        e = errors[mask]
        print(f"{name:<30} {mask.sum():>6} {np.mean(np.abs(e)):>10.2f} {np.sqrt(np.mean(e**2)):>10.2f}")
```

```
Category                       Count        MAE       RMSE
----------------------------------------------------------
Hydrogen and Carbon (H+C)      17639       0.84       1.24
H+C+Nitrogen                   32299       1.13       1.52
H+C+Oxygen                     25244       0.84       1.14
H+C+Nitrogen+Oxygen            11307       1.28       1.64
```

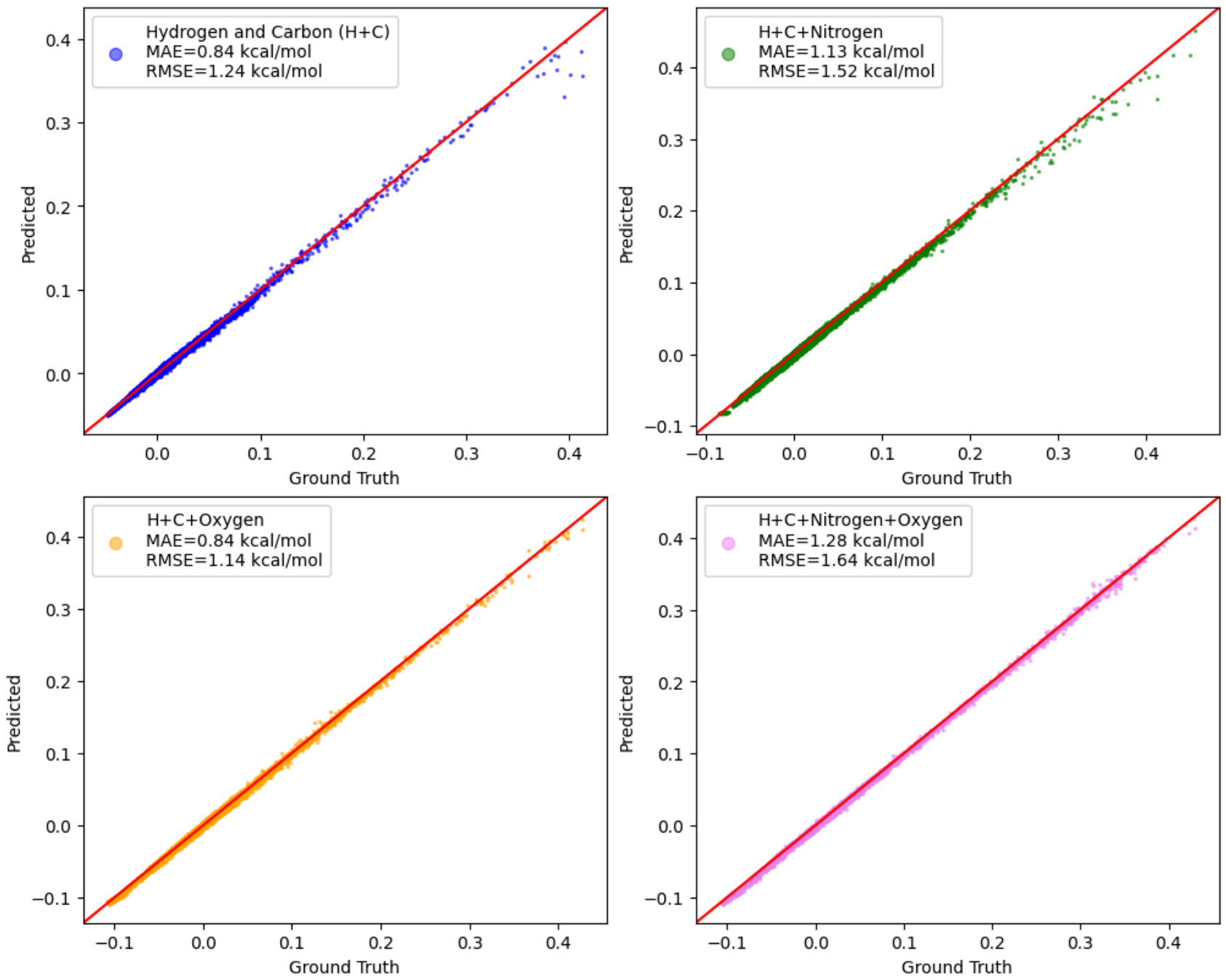
In [19]:
```
fig, axes = plt.subplots(2, 2, figsize=(10, 8), constrained_layout=True)
for ax, (name, mask) in zip(axes.flatten(), categories.items()):
    e = errors[mask]
```

```
    mae  = np.mean(np.abs(e))
    rmse = np.sqrt(np.mean(e ** 2))
    std  = np.std(e)
    ax.hist(e, bins=50, edgecolor='black', alpha=0.7,
            label=f"n={mask.sum()}\nMAE={mae:.2f} kcal/mol\nRMSE={rmse:.2f} kcal/mol\nSTD={std:.2f} kcal/mol")
    ax.axvline(0, color='red', linestyle='--')
    ax.set_title(name)
    ax.set_xlabel("Error (kcal/mol)")
    ax.set_ylabel("Count")
    ax.legend(fontsize=8)
```


Hydrogen and Carbon (H+C)
n=17639
MAE=0.84 kcal/mol
RMSE=1.24 kcal/mol
STD=1.24 kcal/mol
Count
Error (kcal/mol)
H+C+Nitrogen
n=32299
MAE=1.13 kcal/mol
RMSE=1.52 kcal/mol
STD=1.26 kcal/mol
Count
Error (kcal/mol)
H+C+Oxygen
n=25244
MAE=0.84 kcal/mol
RMSE=1.14 kcal/mol
STD=1.11 kcal/mol
Count
Error (kcal/mol)
H+C+Nitrogen+Oxygen
n=11307
MAE=1.28 kcal/mol
RMSE=1.64 kcal/mol
STD=1.24 kcal/mol
Count
Error (kcal/mol)

In [20]:

```
colors = {'Hydrogen and Carbon (H+C)': 'blue', 'H+C+Nitrogen': 'green', 'H+C+Oxygen': 'orange', 'H+C+Nitrogen+Oxygen
fig, axes = plt.subplots(2, 2, figsize=(10, 8), constrained_layout=True)

for ax, (name, mask) in zip(axes.flatten(), categories.items()):
    mae  = np.mean(np.abs(all_true[mask] - all_pred[mask])) * hartree2kcalmol
    rmse = np.sqrt(np.mean((all_true[mask] - all_pred[mask]) ** 2)) * hartree2kcalmol
    ax.scatter(all_true[mask], all_pred[mask], s=2, alpha=0.5, color=colors[name],
               label=f"{name}\nMAE={mae:.2f} kcal/mol\nRMSE={rmse:.2f} kcal/mol")
    ax.set_xlabel("Ground Truth")
    ax.set_ylabel("Predicted")
    xmin, xmax = ax.get_xlim()
    ymin, ymax = ax.get_ylim()
    vmin, vmax = min(xmin, ymin), max(xmax, ymax)
    ax.set_xlim(vmin, vmax)
    ax.set_ylim(vmin, vmax)
    ax.plot([vmin, vmax], [vmin, vmax], color='red')
    ax.legend(markerscale=5)
```


Hydrogen and Carbon (H+C)
MAE=0.84 kcal/mol
RMSE=1.24 kcal/mol
H+C+Nitrogen
MAE=1.13 kcal/mol
RMSE=1.52 kcal/mol
H+C+Oxygen
MAE=0.84 kcal/mol
RMSE=1.14 kcal/mol
H+C+Nitrogen+Oxygen
MAE=1.28 kcal/mol
RMSE=1.64 kcal/mol
Predicted
Ground Truth

In [ ]:

I chose to do 10 fold cross validation. For 10-fold cross validation, the data was split into 80% training, 10% validation data, and 10% test data. That is, one fold must be reserved for early stopping and the learning rate scheduler, another as the held out testing set, and the other 8 for training. Both early stop and ReduceLROnPlateau need to monitor loss on the validation data to decide when to save the best weights and when to reduce the learning rate. However, I also wanted to evaluate on the original 10% test_data from test_data_loader using the 80/10/10 split. The best model produced between the 10 folds of the 10-fold cross-validation achieved an MAE = 1.01 kcal/mol and an RMSE = 1.38 kcal/mol on the original held-out test set. The same model achieved an MAE of 1.016 kcal/mol and RMSE = 1.38 kcal/mol on its fold held-out test set. The ANI paper had an RMSE of 1.3 kcal/mol.

To explain the gap in error, we must look at the training data. In the ANI paper, the scientist's dataset contained on roughly 17.2 million conformations from roughly 57,951 molecules. Their dataset contained subsets of molecules up to 8 heavy atoms. I trained on only up to 4 heavy atoms. My dataset contained 846,896 conformations. My dataset was roughly 5% the size of their dataset. With fewer molecules, the model is exposed to fewer types of bonds, functional groups, and structures during training. Thus having less information to predict energies of new molecules.

The model architecture in the ANI paper is 768 -> 128 -> 128 -> 64 -> 1. They had 124,033 parameters in their network. They also had a 768-element AEV using 32 radial symmetry functions that probe out from each atom at evenly spaced distances with a cut off of 4.6 angstroms. The angular symmetry function uses 8 distance shifts and 8 angle shifts with a cut off of 3.1 angstroms. Having the atoms H, C, N, and O, the radial part produces 128 elements (4 atom types * 32 shifts). The angular parts produces (10 unique atom type pairs * 8 radial shifts * 8 angular shifts) = 640 elements. This totals to a 768 element AEV. My AEV describes each atom's local environment with roughly half the resolution. My AEV uses 16 radial shifts with a 5.2 angstrom cutoff an 8 angle shifts up to 3.5 angstroms as its cutoff. There are 64 (4 * 16 shifts) radial elements and 320 angular components (10 unique atom type pairs * radial shifts * 8 angular shifts). My architecture is 384 -> 512 -> 256 -> 128 -> 1. My network contains roughly 363, 265 parameters. The ANI paper uses Gaussian activation with an exponential cost function. They reported that this method gives 2-4x lower error than standard approaches. I used ReLU and an MSE loss function. The ANI paper reduced the learning rate 6 times, starting from 0.001. Each time reducing by an order of magnitude after the model has converged under its current learning rate. I took a different approach and used ReduceLRONPlateau with a factor of 0.5 and a patience of 10. That is, when the model sees that the validation loss had no meaningful decrease within 10 epochs, the learning rate is halved.

Overall, my model achieved element category MAEs that are near or below the 1 kcal/mol chemical accuracy threshold. The MAEs are H+C = 0.84 kcal/mol, H+C+N = 1.13 kcal/mol, H+C+O = 0.84 kcal/mol, and H+C+N+O = 1.28 kcal/mol.